

HOMNI

Website Redesign

Stage 3 — Technical Documentation

B2B showcase website and content-management dashboard

React + Vite · NestJS · MySQL 8 · TypeORM

Team: Leny BLEE · Ethane ZIMMERMANN · Tom MARCHAL

Holberton School · Tutored Group Project · 12 weeks

Client: Homni — Montauban / Toulouse, Occitanie

Document version: 1.0 · May 2026

— *English version* —

0. Introduction

0.1 Document purpose

This document is the Stage 3 deliverable for the Homni website redesign — a tutored group project conducted at Holberton School. It translates the project objectives defined in Stage 1 (team formation and MVP selection) and Stage 2 (high-level plan, 12-week schedule and SMART objectives) into a concrete technical plan: user stories, system architecture, data model, sequence diagrams, API surface and the processes (SCM, QA, deployment) that will govern the implementation.

It is the single source of truth for the build phase. Any technical decision taken during development must either be consistent with what is described here or be tracked as an explicit deviation through a pull request that updates this document.

0.2 The client and the product in one paragraph

Homni is a B2B service provider based in Montauban and Toulouse that offers four complementary services to companies in Occitanie: IP telephony, IT services (network, helpdesk, cybersecurity), printing solutions and dynamic display. Their existing website does not reflect the company's positioning and offers no path for prospects to qualify themselves and request an appointment. The redesign replaces it with a modern responsive showcase site clearly presenting the four service areas, a public blog for content marketing, and a structured contact path. It also gives Homni an admin dashboard so the editorial team can publish blog articles autonomously, without depending on a developer.

0.3 MVP scope covered

This document covers the agreed Stage-1 MVP: the public showcase site (7 key pages — home, four service pages, about, blog list + detail, contact), the admin dashboard for blog-article CRUD only (other editorial features are out of scope for this iteration), JWT-based authentication for admin users, the contact form including GDPR consent, and the deployment pipeline. The data model is intentionally narrow: 6 tables. The roadmap may stage the build in phases, but the design must support the entire MVP from day one because schema and API are hard to refactor once the public site is live and indexed by search engines.

1. User Stories and Mockups

1.1 Personas

Camille — the SME owner (prospect). Runs a 35-person company in Montauban that is about to lose its analogue telephony line. Doesn't know the difference between SIP trunking and a hosted PBX. Needs the site to explain the offer in plain French, to feel like Homni is a reliable local partner, and to book an appointment in under two minutes.

Marc — the IT director at a 200-person company (informed prospect). Comparing 3 providers for an outsourcing renewal. Already knows the technical vocabulary. Wants concrete references, recent blog articles that prove Homni's expertise, and clear contact information for a technical pre-sales call.

Sophie — Homni's marketing manager (admin user). In charge of publishing 2 to 3 blog articles per month. Not a developer; used to WordPress-style editors. Needs to draft, save, preview, attach a cover image and publish without asking the agency.

Pierre — Homni's commercial director (admin user, light). Logs in once or twice a week to read the contact messages received and reply by email. Needs the dashboard to be obvious and fast on a laptop browser.

1.2 Prioritized user stories (MoSCoW)

Stories carry a unique ID for traceability against API endpoints, sequence diagrams and test cases (US-xx).

Must Have

ID	Persona	Story	Priority
US-01	Camille / Marc	As a visitor, I want to land on a homepage that clearly states what Homni does and for whom, so I can decide in 10 seconds whether to stay.	Must
US-02	Camille / Marc	As a visitor, I want a dedicated page for each of the 4 services (telephony, IT, printing, display) so I can dive into the one relevant to me.	Must
US-03	Camille	As a visitor, I want a contact form with a clear subject selector so I can request information without picking up the phone.	Must
US-04	Camille	As a visitor, I want the site to load and feel fast on my office desktop and on my phone so I do not bounce.	Must
US-05	Camille / Marc	As a visitor, I want the site to be fully responsive so I can browse from any device including a 5-year-old tablet.	Must
US-06	Marc	As an informed visitor, I want to read a blog with categorised technical articles so I can assess Homni's expertise.	Must
US-	Marc	As a visitor, I want each blog article to have a clean	Must

ID	Persona	Story	Priority
07		readable layout with an image, title, body and publication date.	
US-08	Sophie	As an editor, I want to sign in to a private dashboard with my email and password so I can manage content.	Must
US-09	Sophie	As an editor, I want to create, edit, save as draft and publish blog articles so I can ship content autonomously.	Must
US-10	Sophie	As an editor, I want to attach a cover image to each article so the blog list looks professional.	Must
US-11	Sophie / Pierre	As an editor, I want to choose a category for each article (telephony, IT, printing, display) so the public blog can be filtered.	Must
US-12	All visitors	As a visitor, I want a GDPR-compliant consent banner and a privacy page so my data is handled lawfully.	Must

Should Have

ID	Persona	Story	Priority
US-13	Pierre	As a commercial, I want to see the contact-form messages in the dashboard so I do not have to dig through my inbox.	Should
US-14	Sophie	As an editor, I want a preview of my article before publishing so I can check the rendering.	Should
US-15	Marc	As a visitor, I want to filter blog articles by category so I can quickly find what is relevant to me.	Should
US-16	Pierre	As a commercial, I want to mark a contact message as read or archived so the dashboard stays tidy.	Should
US-17	All visitors	As a visitor, I want a sitemap.xml and proper meta tags so the site is correctly indexed by Google.	Should

Could Have

ID	Persona	Story	Priority
US-18	Sophie	As an editor, I want to schedule an article for future publication so I can prepare a content calendar.	Could
US-19	Pierre	As a commercial, I want a basic dashboard chart of monthly contact messages so I can track lead volume.	Could
US-20	All visitors	As a visitor, I want to subscribe to a newsletter from the blog so I receive new articles.	Could

ID	Persona	Story	Priority
US-21	Sophie	As an editor, I want a media library to reuse images across articles so I do not re-upload the same file twice.	Could

Won't Have (this iteration)

- US-22 — Online quote generation. Sales cycle is consultative; a quote requires a phone call. Out of MVP.
- US-23 — Client portal (invoices, tickets). Would duplicate Homni's existing internal tools; not in scope.
- US-24 — Multilingual site (English version). Homni's market is Occitanie; deferred.
- US-25 — Inline article commenting. Moderation cost is disproportionate for the expected traffic.

1.3 Mockups — main screens

Three screens carry the bulk of the experience: the public homepage (first impression), the blog list (content-marketing surface) and the admin article editor (the editor's main workspace). Wireframes are intentionally low-fidelity to focus the team on layout and information hierarchy rather than visual design — the final visual style will be defined in Figma at the beginning of Stage 4.

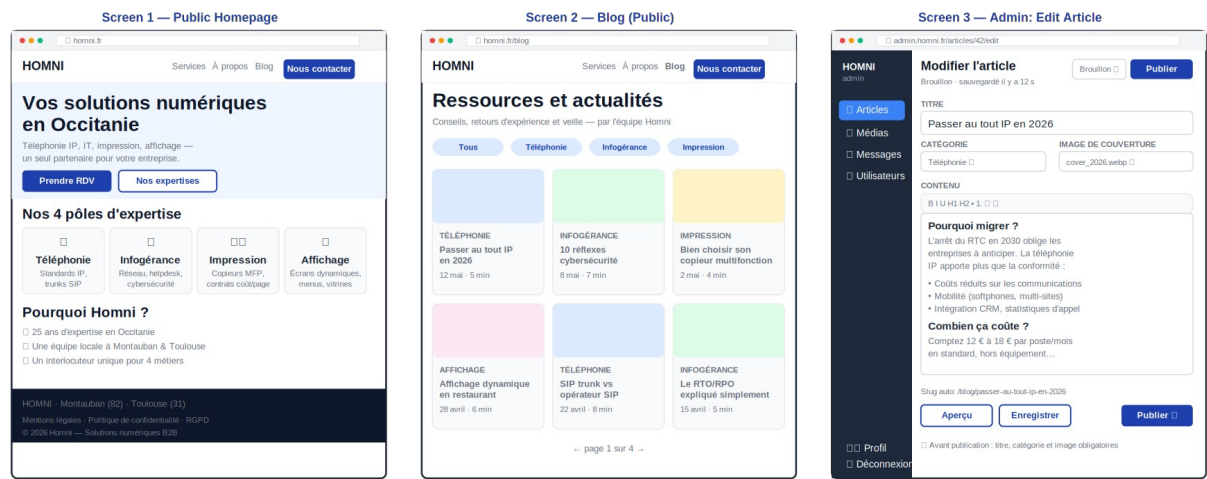


Figure 1 — Wireframes of the three main screens: Public homepage (US-01 to US-05), Public blog list with category filters (US-06, US-15), Admin article editor (US-09, US-10, US-11, US-14).

Screen 1 — Public homepage

- Top navigation: logo + 3 links (Services, About, Blog) + a contrasted "Contact us" call-to-action.
- Hero block stating the value proposition in one short sentence with two CTAs: "Book an appointment" (primary) and "Our expertise" (secondary).
- "4 areas of expertise" row — one card per service, each navigating to its dedicated page (US-02).
- Trust block listing 3 to 4 reassuring facts (years of experience, local presence, single point of contact).
- Footer with legal links (mentions légales, privacy policy, GDPR) — mandatory for US-12.

Screen 2 — Public blog list

- Page title and one-sentence subtitle setting expectations ("Resources and news from the Homni team").
- Category filter pills (Tous, Téléphonie, Infogérance, Impression, Affichage) — US-15.
- Responsive grid of article cards (3 per row on desktop, 2 on tablet, 1 on mobile). Each card shows category, title, date and an estimated reading time.
- Pagination at the bottom (no infinite scroll — better for SEO and for older browsers).

Screen 3 — Admin article editor

- Persistent left sidebar: Articles, Médias, Messages, Utilisateurs.
- Editor header: title, save status (auto-save every 30 s), publication-status selector, primary "Publier" button.
- Form fields: title, category, cover image (US-10), then a rich-text editor for the body. The slug is generated from the title and shown read-only.
- Bottom action bar: Aperçu (preview, US-14), Enregistrer (save as draft), Publier (publish).
- Inline validation message reminding the editor that title, category and cover image are required before publishing.

2. System Architecture

2.1 High-level diagram

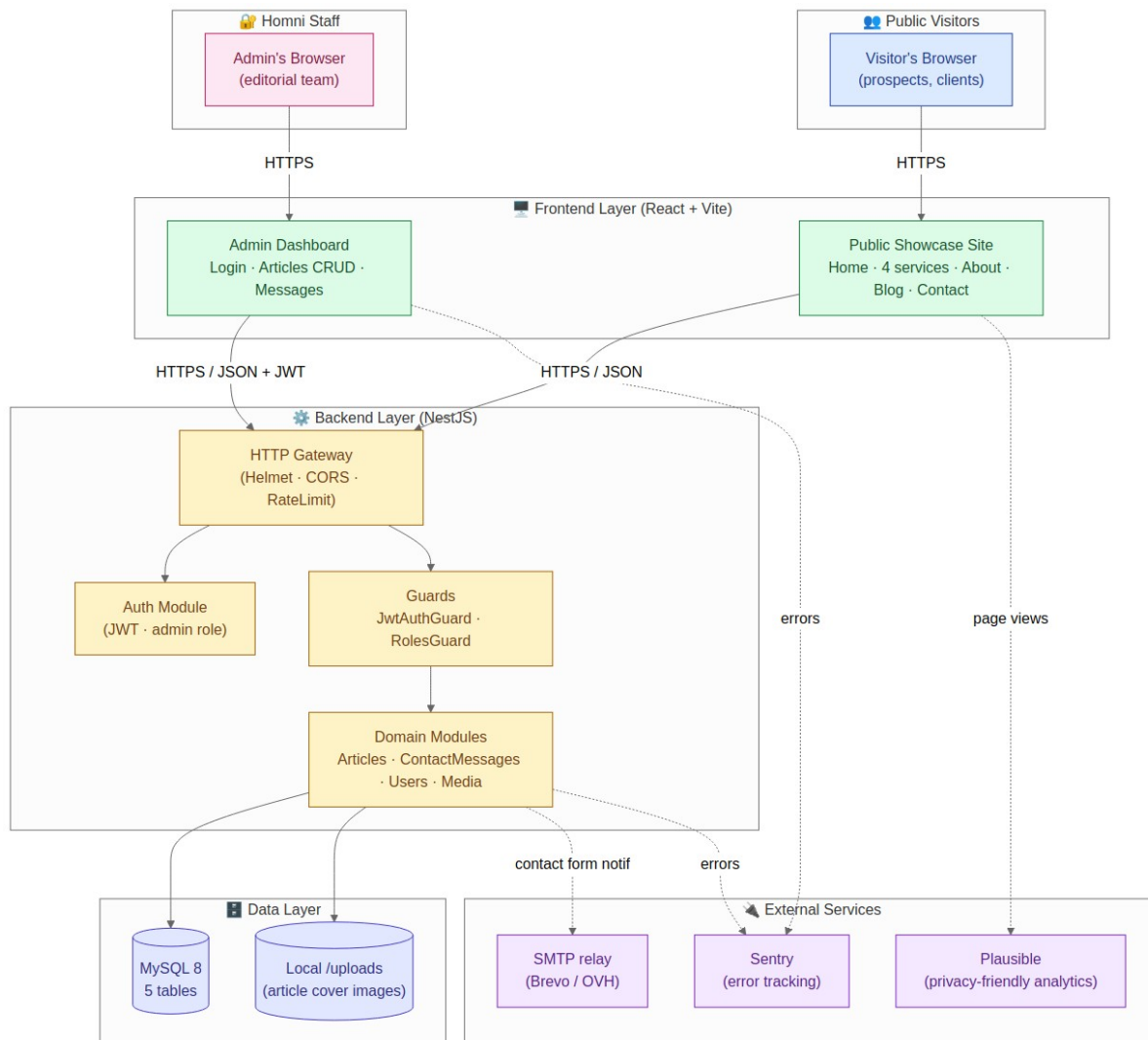


Figure 2 — Component diagram. Solid arrows = synchronous HTTP / JSON. Dotted arrows = asynchronous or deferred flows (contact-form notifications, error reporting, analytics).

2.2 Layer responsibilities

Frontend layer — React + Vite

The project is a TypeScript monorepo with two React applications sharing a UI-component package. The public showcase site is server-rendered at build time (static export through Vite + a small pre-rendering step for the blog) so every page is delivered as plain HTML — critical for SEO and Lighthouse scores (US-04, US-17). The admin dashboard is a classic single-page application served behind authentication and is not indexed.

Backend layer — NestJS

NestJS structures the API around domain modules (Auth, Users, Articles, Categories, Media, ContactMessages) with explicit dependency injection. Cross-cutting concerns live at the gateway: Helmet for HTTP security headers, a CORS policy restricted to the two known frontend origins, and a rate limiter (stricter on /contact-messages to absorb spam, looser on read endpoints). Two global guards (JwtAuthGuard, RolesGuard) enforce authentication and role-based authorization before any controller runs.

Data layer — MySQL 8 and local filesystem

MySQL 8 stores the durable domain data. The schema (6 tables, see section 3) is managed through TypeORM migrations and is never modified directly in production. Article cover images are stored on the server's local filesystem under /uploads with deterministic UUID-based filenames; a backup cron job rsyncs this folder daily to an off-site bucket. A managed object store (S3-compatible) is a deferred decision: it is not justified at the expected article volume (< 200 images / year) and adds operational complexity.

External services

- SMTP relay (Brevo or OVH SMTP) — used only to notify the Homni team of new contact-form submissions. No transactional email is sent to visitors as part of the MVP.
- Sentry — server-side and client-side error tracking with release tagging. Free tier is sufficient at the expected traffic.
- Plausible Analytics — privacy-friendly page-view analytics. Chosen over Google Analytics to avoid the GDPR consent banner complication for purely aggregated analytics.

2.3 Request walkthrough

A typical write request — "publish an article" — traces a clean path through the layers:

1. The admin dashboard sends POST /articles with the admin's JWT in the Authorization header.
2. The gateway applies Helmet headers and validates that the request origin is the admin dashboard.
3. JwtAuthGuard verifies the token signature and attaches { userId, role } to the request.
4. RolesGuard checks that the role is admin or editor — required for any write on /articles.
5. ArticlesController validates the body (class-validator DTO) and delegates to ArticlesService.
6. The service generates the slug, runs the INSERT inside a transaction, and returns the created row.
7. The response is serialized through a Nest interceptor that strips internal fields and returns 201 with the new article.

A typical read request — "render the blog list" — is shorter: the public site fetches GET /articles?status=published&category=... without authentication, the gateway applies a generous rate limit, and the response is cached for 5 minutes by an HTTP Cache-Control header. Cache invalidation is implicit: at the next publish, the new content appears at most 5 minutes later — acceptable for a marketing site.

3. Components, Classes and Database Design

3.1 Entity-Relationship Diagram

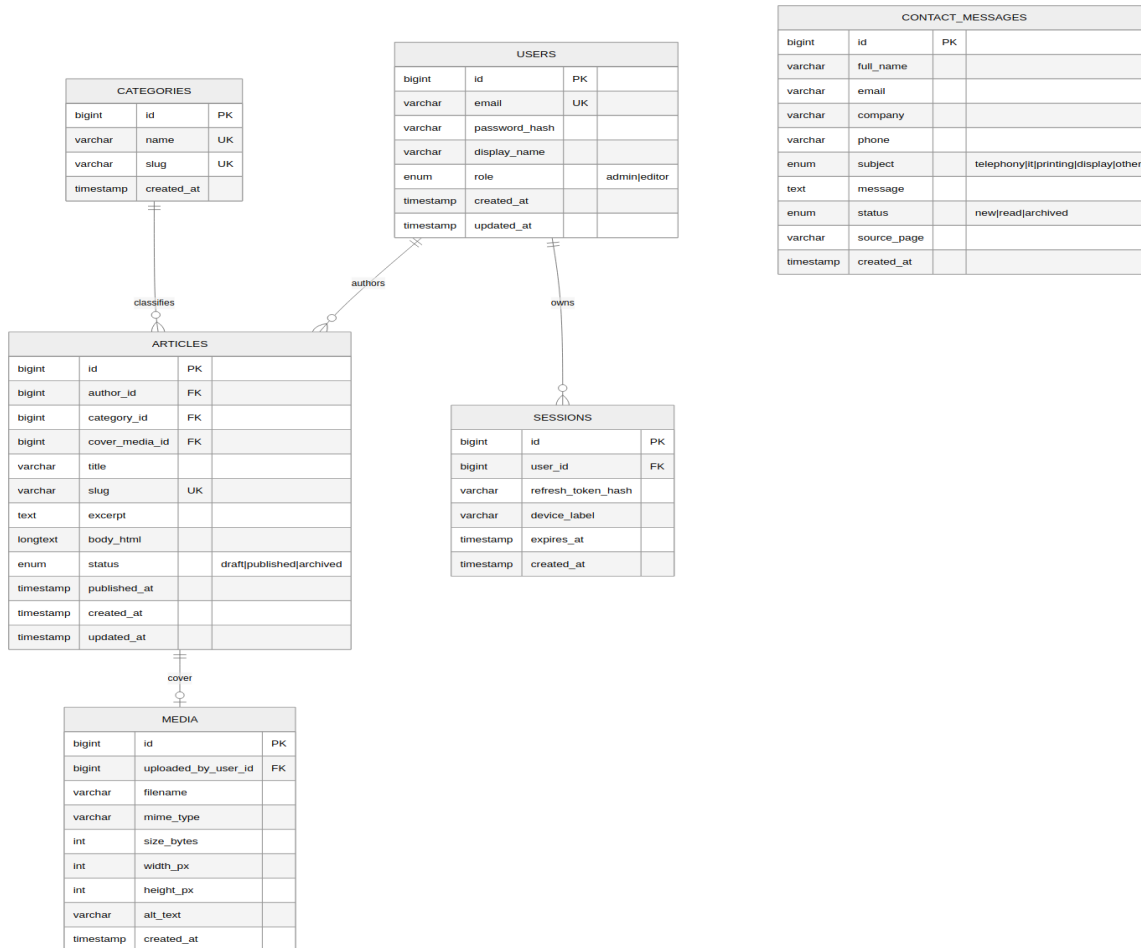


Figure 3 — Entity-Relationship Diagram. 6 tables, all using BIGINT surrogate primary keys and InnoDB. Composite uniques and indexes are documented in section 3.4.

3.2 Backend class structure

Each domain module follows the same shape — Module / Controller / Service / Repository / Entity / DTOs. Below are the key classes with their main responsibilities.

Class	Responsibility
UserService	Account creation (admin-only), profile update, password change. Wraps password hashing (argon2id). Roles are admin (full access) and editor (no user management).
AuthService	Issues access tokens (15 min) and refresh tokens (30 days, hashed in DB). Verifies credentials. Triggers password reset emails (deferred to v1.1).
JwtAuthGuard	Global guard. Validates JWT signature and freshness, attaches user context

Class	Responsibility
	to the request.
RolesGuard	Param-aware guard. For routes annotated with @Roles('admin' 'editor'), checks the request user's role.
ArticlesService	CRUD on articles. Generates the URL slug from the title (slugify + collision check). Enforces required fields at publication (title, body, category, cover).
CategoriesService	Read-only catalogue exposing the four fixed business categories. Seeded in a migration; not editable through the dashboard.
MediaService	Stores uploaded files under /uploads, validates MIME type and dimensions, writes a media row pointing to the file. Files are served by Nest as static assets behind a CDN-friendly URL.
ContactMessagesService	Persists contact-form submissions, runs server-side validation and a honeypot check, fires an SMTP notification asynchronously.
NotificationService	Thin wrapper around nodemailer that the ContactMessagesService and (later) password-reset flows use. Email content is templated.

3.3 Database schema (MySQL 8)

All tables use InnoDB with utf8mb4 / utf8mb4_0900_ai_ci. Surrogate BIGINT primary keys are auto-incremented. Foreign keys cascade on DELETE only where the relation is ownership-style (an article without an existing author is meaningless and is blocked by ON DELETE RESTRICT; an article without a cover image keeps the row, since SET NULL is the right behaviour).

Table: users

Column	Type	Constraints	Notes
id	BIGINT	PK, AUTO_INCREMENT	Surrogate key.
email	VARCHAR(255)	UNIQUE, NOT NULL	Lowercased on write.
password_hash	VARCHAR(255)	NOT NULL	argon2id.
display_name	VARCHAR(60)	NOT NULL	Shown as the article byline.
role	ENUM('admin','editor')	NOT NULL	admin = full access; editor = articles + media only.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

Column	Type	Constraints	Notes
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	

Table: sessions

Column	Type	Constraints	Notes
id	BIGINT	PK	
user_id	BIGINT	FK → users.id, ON DELETE CASCADE	
refresh_token_hash	VARCHAR(255)	NOT NULL	argon2id hash; indexed for lookups.
device_label	VARCHAR(120)	NULL	User-agent summary.
expires_at	TIMESTAMP	NOT NULL	30-day rolling window.
created_at	TIMESTAMP	DEFAULT NOW	

Table: categories

Column	Type	Constraints	Notes
id	BIGINT	PK	
name	VARCHAR(60)	UNIQUE, NOT NULL	Téléphonie, Infogérance, Impression, Affichage.
slug	VARCHAR(60)	UNIQUE, NOT NULL	URL fragment: telephonie, infogerance, impression, affichage.
created_at	TIMESTAMP	DEFAULT NOW	

Table: media

Column	Type	Constraints	Notes
id	BIGINT	PK	
uploaded_by_user_id	BIGINT	FK → users.id, NULL	Null if the uploader is deleted; image stays.

Column	Type	Constraints	Notes
filename	VARCHAR(255)	UNIQUE, NOT NULL	Deterministic UUID-based filename on disk.
mime_type	VARCHAR(80)	NOT NULL	image/webp, image/jpeg, image/png only.
size_bytes	INT	NOT NULL	Max 5 MB enforced at upload.
width_px	INT	NOT NULL	Used by the front-end for layout shifting prevention.
height_px	INT	NOT NULL	
alt_text	VARCHAR(255)	NOT NULL	Mandatory for accessibility (WCAG).
created_at	TIMESTAMP	DEFAULT NOW	

Table: articles

Column	Type	Constraints	Notes
id	BIGINT	PK	
author_id	BIGINT	FK → users.id, ON DELETE RESTRICT	Articles outlive uploaders, not authors.
category_id	BIGINT	FK → categories.id, ON DELETE RESTRICT	
cover_media_id	BIGINT	FK → media.id, NULL ON DELETE SET NULL	Optional but required by validation before publishing.
title	VARCHAR(180)	NOT NULL	
slug	VARCHAR(200)	UNIQUE, NOT NULL	Generated; lowercased; URL-safe.
excerpt	VARCHAR(280)	NOT NULL	Used in the blog list and meta description.
body_html	LONGTEXT	NOT NULL	Sanitised HTML produced by the rich-text editor.
status	ENUM('draft','published','archived')	DEFAULT 'draft'	Index for filtered list queries.
published_at	TIMESTAMP	NULL	Set when status flips to

Column	Type	Constraints	Notes
			published.
created_at	TIMESTAMP	DEFAULT NOW	
updated_at	TIMESTAMP	ON UPDATE NOW	

Table: contact_messages

Column	Type	Constraints	Notes
id	BIGINT	PK	
full_name	VARCHAR(120)	NOT NULL	
email	VARCHAR(255)	NOT NULL	Validated; lowercased.
company	VARCHAR(120)	NULL	
phone	VARCHAR(40)	NULL	
subject	ENUM('telephony','it','printing','display','other')	NOT NULL	Matches the contact-form selector.
message	TEXT	NOT NULL	Max 2000 chars at the form level.
status	ENUM('new','read','archived')	DEFAULT 'new'	Drives the dashboard inbox view (US-13, US-16).
source_page	VARCHAR(120)	NULL	Page from which the form was submitted, for attribution.
created_at	TIMESTAMP	DEFAULT NOW	

3.4 Key indexes (beyond primary keys)

- users.email — UNIQUE.
- articles.slug — UNIQUE.
- articles (status, published_at DESC) — drives the public blog list query.
- articles (category_id, status, published_at DESC) — drives filtered category queries.
- sessions.refresh_token_hash — non-unique index for fast lookup.
- contact_messages (status, created_at DESC) — drives the dashboard inbox.

3.5 Front-end component tree

Two React applications share a UI-component package. Routing on the public site uses file-based routes (one component per page); the admin dashboard uses a nested router with a persistent shell.

Public site

Component	Role
RootLayout	Header (logo + nav + contact CTA), main outlet, footer, GDPR consent banner.
HomePage	Hero, four-service row, trust block, latest articles teaser, final CTA.
ServicePage	One per service: hero, problem statement, offer, mini case study, CTA. Four instances driven by route param.
AboutPage	Company story, team, geographic footprint.
BlogListPage	Category filter, paginated article grid (US-06, US-15).
BlogArticlePage	Hero with cover image, title, byline, body, share buttons (US-07).
ContactPage	Contact form (US-03) with subject selector, GDPR consent checkbox, success / error states.
LegalPages	Mentions légales, Politique de confidentialité, Cookies. Plain content pages required by law.

Admin dashboard

Component	Role
AdminShell	Sidebar + topbar + outlet. Enforces an authenticated session at the route level.
LoginPage	Email + password form (US-08). Redirects authenticated users to /articles.
ArticlesListPage	Sortable, filterable table of all articles (status, category, author). Bulk actions.
ArticleEditorPage	Editor for create + edit (US-09). Rich-text editor, slug preview, cover-image picker.
MediaLibraryPage	Grid of uploaded media with the option to upload, rename alt text, delete unused files.
ContactInboxPage	Inbox of contact-form messages (US-13). Mark as read / archived (US-16).
UsersPage	Admin-only. CRUD on users (invite by email, set role).

4. High-Level Sequence Diagrams

Three flows have been selected as critical to validate. Together they exercise authentication, an authenticated write path with file upload, and an unauthenticated write path open to the public — the three categories of risk in the architecture.

4.1 Admin login (US-08)

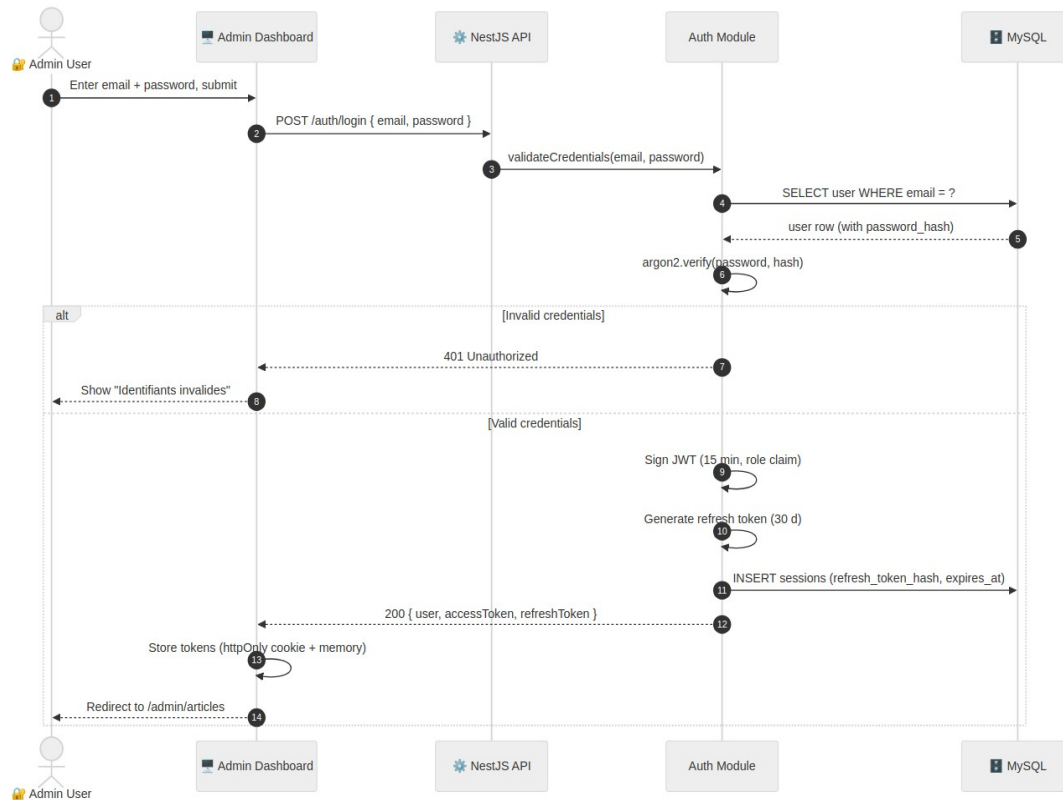


Figure 4 — Admin login flow. Both branches (invalid + valid) are documented so the failure path is testable.

Notes on the flow:

- Credentials are verified server-side only. The hash comparison uses `argon2.verify`, which is constant-time and resistant to timing attacks.
- The JWT carries the user id and role as claims, signed with HS256. The dashboard uses the role claim to render the right navigation items, but the backend never trusts it without re-verifying through RolesGuard.
- Refresh tokens are stored hashed (`argon2id`). The raw token is sent only once in the login response and lives in an `httpOnly` cookie on the dashboard.
- A login attempt limit (5 failures per IP per 15 min) is applied at the gateway to mitigate brute-force attempts.

4.2 Publish a blog article (US-09, US-10, US-11)

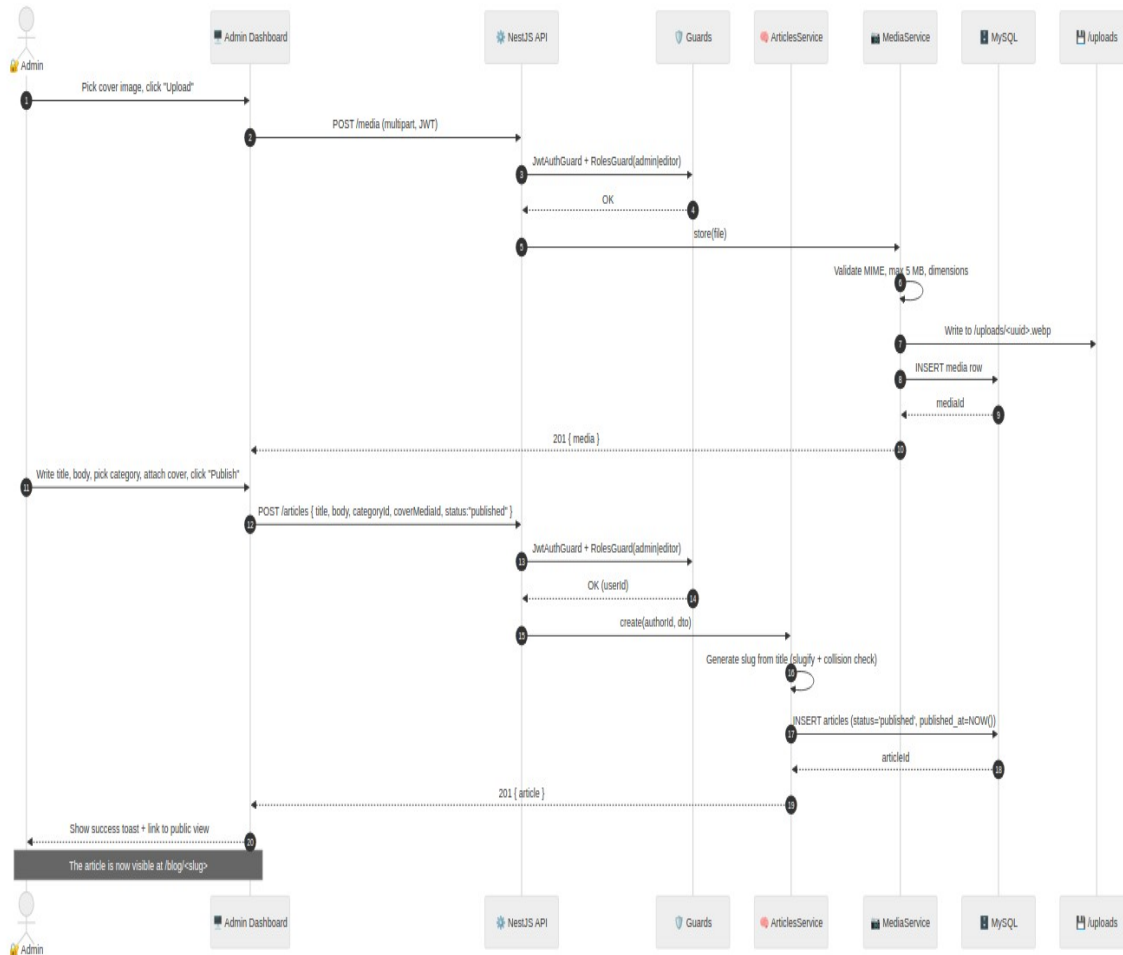


Figure 5 — Two-step flow: media upload first, then article creation. The two steps are decoupled so the same image can be reused on multiple articles in a future iteration.

Notes on the flow:

- File uploads are validated for MIME, size and dimensions on the server, regardless of any client-side validation.
- Uploaded files are renamed to a UUID at storage time so the original filename never appears in the public URL.
- Slugs are generated by the service, not by the client, to guarantee uniqueness and URL safety.
- Once status flips to published, the article appears on the public site at the next cache refresh (≤ 5 minutes).

4.3 Submit the contact form (US-03)

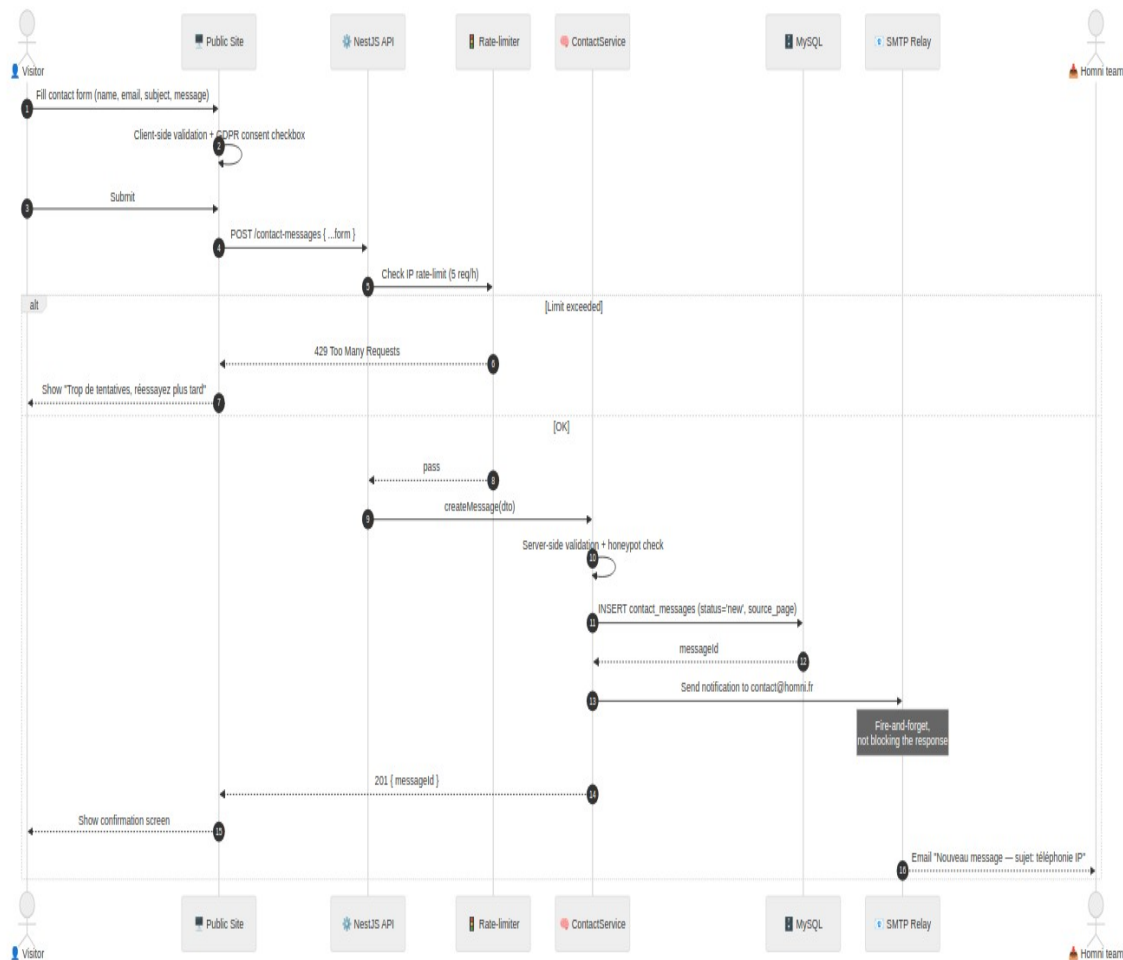


Figure 6 — Contact form submission. The rate-limit and the honeypot field together absorb most spam without affecting legitimate visitors.

- The honeypot field is a hidden input that humans never fill. A non-empty value triggers a silent rejection (200 OK is returned to avoid signalling the trap to bots).
- SMTP notification is fire-and-forget so the visitor's request returns quickly. Email failures are logged to Sentry but do not propagate to the user.
- The source_page field is populated by the client from the referrer so the commercial team can correlate leads with the entry-point page.

5. API Specifications

5.1 External APIs consumed

Provider	Purpose	Justification
SMTP relay (Brevo or OVH)	Notifying Homni of new contact messages	Both have generous free or low-cost tiers covering our expected volume (<200 messages/month). SMTP keeps Homni in control of the destination mailbox without locking us into a vendor API.
Sentry	Error tracking (backend + frontends)	Free tier sufficient for MVP. Release tagging integrates with our deployment script so each error references the exact commit.
Plausible Analytics	Privacy-friendly page-view analytics	Self-hostable, no personal data, no cookie banner required for analytics — simplifies GDPR compliance. Chosen over Google Analytics for that reason.

5.2 Internal REST API — conventions

- Base URL: `https://api.homni.fr/v1`
- All requests and responses use `application/json; charset=utf-8` (except media upload, which uses `multipart/form-data`).
- Authenticated routes require `Authorization: Bearer <accessToken>`. Public routes: `/articles` (read), `/categories`, `/contact-messages` (POST), `/auth/login`, `/auth/refresh`.
- Errors follow an RFC 7807-like shape: `{ statusCode, error, message, code }`.
- Pagination is offset-based on the dashboard (`page + perPage`) since data is small; the public blog list also uses offset for clean SEO-friendly URLs (`?page=2`).
- Date-times are ISO-8601 with timezone (the server is UTC). The client converts for display.
- Versioning is in the path (`/v1/...`). A breaking change increments to `/v2` and runs in parallel for at least one release cycle.

5.3 Endpoint catalogue

The surface is intentionally narrow — about 30 endpoints across 6 modules — because the MVP scope is bounded to blog and contact. The full input/output contracts are maintained in `openapi.yaml` and rendered through Swagger UI at `/docs` in non-production environments.

Auth

Method	Path	Purpose / response
POST	<code>/auth/login</code>	Email + password sign-in. → 200 { user, accessToken, refreshToken }.
POST	<code>/auth/refresh</code>	Trade a refresh token for a new pair. → 200 { accessToken, refreshToken }.

Method	Path	Purpose / response
POST	/auth/logout	Revoke the current refresh token. → 204.
GET	/auth/me	Return the authenticated user. → 200 { user }.

Users (admin-only)

Method	Path	Purpose / response
GET	/users	List all admin users. → 200 [{ id, email, displayName, role }].
POST	/users	Invite a new user (sends a reset-password email). → 201 { user }.
PATCH	/users/:id	Update display name or role. → 200 { user }.
DELETE	/users/:id	Soft-delete a user (cannot delete the last admin). → 204.
PATCH	/users/me/password	Change own password (requires current). → 204.

Categories (read-only catalogue)

Method	Path	Purpose / response
GET	/categories	Public. Returns the 4 fixed categories. → 200 [{ id, name, slug }].

Articles

Method	Path	Purpose / response
GET	/articles? status=&categorySlug=&page=& perPage=	Public when status=published; otherwise admin. → 200 { items, total, page }.
GET	/articles/:slug	Public lookup by slug for the article page. → 200 { article }.
GET	/articles/id/:id	Admin lookup by id (used by the editor). → 200 { article }.
POST	/articles	Create an article (draft by default). → 201 { article }.
PATCH	/articles/:id	Update fields and/or transition status. → 200 { article }.
POST	/articles/:id/publish	Convenience endpoint: status='published', published_at=NOW. → 200 { article }.

Method	Path	Purpose / response
POST	/articles/:id/archive	Archive an article — hidden from public list. → 200 { article }.
DELETE	/articles/:id	Hard-delete (admin only; editor can only archive). → 204.

Media

Method	Path	Purpose / response
GET	/media?page=&perPage=	List uploaded media (admin). → 200 { items, total, page }.
POST	/media	Upload a new media file (multipart). → 201 { media }.
PATCH	/media/:id	Update alt text. → 200 { media }.
DELETE	/media/:id	Delete (refused if referenced by any article). → 204 or 409.

Contact messages

Method	Path	Purpose / response
POST	/contact-messages	Public submission from the contact form. → 201 { messageId }.
GET	/contact-messages?status=	Admin inbox view (US-13). → 200 { items }.
PATCH	/contact-messages/:id	Update status to read or archived (US-16). → 200 { message }.
DELETE	/contact-messages/:id	Hard-delete a message (admin only). → 204.

5.4 Example request / response

POST /contact-messages — a visitor submits the contact form:

```
POST /v1/contact-messages
Content-Type: application/json

{
  "fullName": "Camille Durand",
  "email": "c.durand@exemple.fr",
  "company": "Atelier Durand SAS",
  "phone": "+33 5 63 00 00 00",
  "subject": "telephony",
  "message": "Bonjour, nous souhaitons migrer notre standard en IP avant la fin de l'année.",
}
```

```
"sourcePage": "/services/telephonie",  
"website": ""  
}
```

Response (the website field is the honeypot; an empty value is normal):

HTTP/1.1 201 Created

Content-Type: application/json

```
{  
  "messageId": 318,  
  "createdAt": "2026-05-25T14:09:51.027Z"  
}
```

6. SCM and QA Plans

6.1 Source-control strategy

The project uses a trimmed Git Flow with two long-lived branches and short-lived feature, release and hotfix branches. This was chosen over trunk-based development because (a) the team is composed of students with overlapping schedules — a stable main branch protects the deliverable while parallel work continues on develop — and (b) the client wants discrete, tagged versions (v1.0.0, v1.1.0...) tied to identifiable scope, which release branches model naturally.

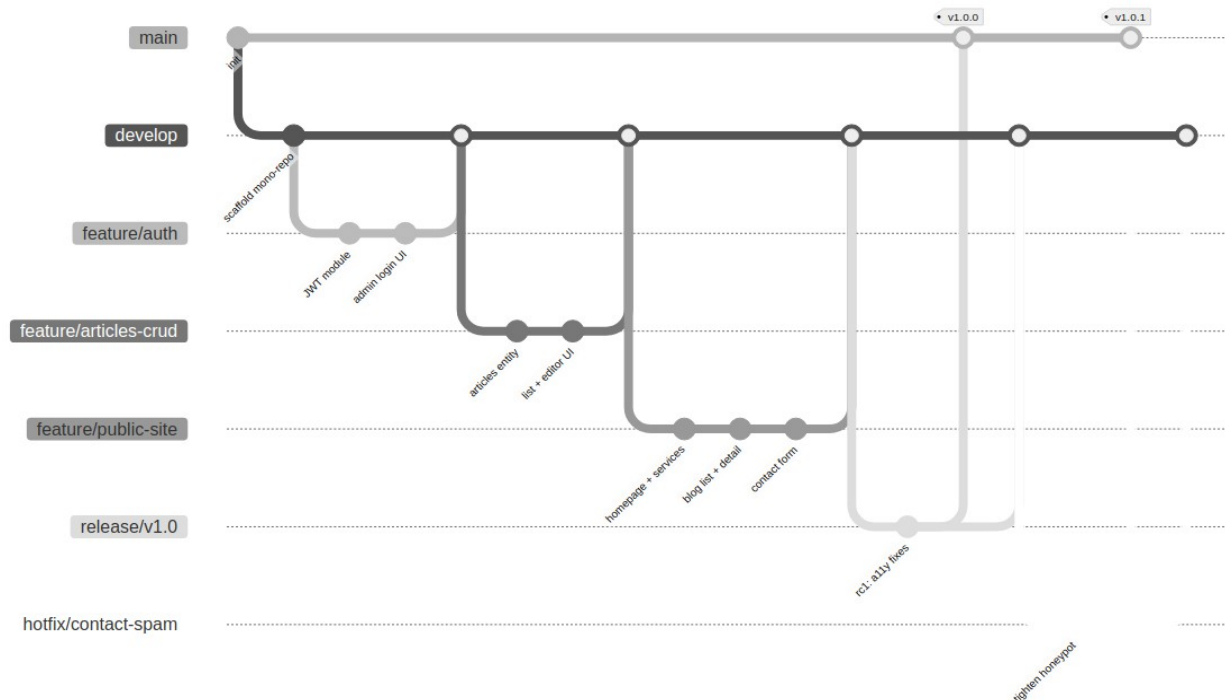


Figure 7 — Branching model. *main* is always equal to the latest tagged release; *develop* is the integration branch; feature branches are short-lived (≤ 1 week); hotfix branches branch from *main* and merge into both *main* and *develop*.

Branches

- *main* — production-ready code. Every merge is tagged (vX.Y.Z) and deploys to production.
- *develop* — integration branch. All feature branches merge here. The staging environment auto-deploys from *develop*.
- *feature/<short-slug>* — one branch per user story or technical task. Branched from *develop*, merged via reviewed PR.
- *release/vX.Y.Z* — opened a few days before a client demo for last-minute polish. Merged into *main* and *develop*.
- *hotfix/<slug>* — emergency fixes branched from *main*, merged into *main* and *develop*.

Commit and PR conventions

- Conventional Commits format: type(scope): subject — types are feat, fix, chore, docs, refactor, test, ci, perf.

- PRs require at least one passing CI build, a green test suite, one reviewer approval, and a self-review checklist (no commented-out code, no console.log, migrations included where relevant).
- PR titles reference the user-story ID — e.g. "feat(articles): publish flow (US-09)".
- Squash-merge is the default to keep develop history linear; release / hotfix merges into main use merge commits to preserve tags.
- Branch protection on main: no direct pushes, signed commits enforced.

6.2 Quality-assurance strategy

Quality is enforced at three levels — static, automated and manual — each chosen to catch a different class of bug at the cheapest possible stage.

Static layer (catches bugs without running code)

- TypeScript in strict mode on both backend and frontend.
- ESLint + Prettier with a shared config across the monorepo. CI fails on lint errors, not just warnings.
- class-validator on every DTO. Invalid bodies are rejected at the gateway with a 400 before reaching a service.
- Database migrations checked into Git; CI verifies that the local DB schema dropped and re-migrated matches the schema dump from develop.
- axe-core run inside the public-site test suite to fail PRs that introduce WCAG violations on the 7 key pages (supports the accessibility objective from Stage 2).

Automated tests

Layer	Tool	What is tested
Backend unit	Jest	Service methods (slug generation, role enforcement, status transitions). Repository layer mocked. Target: ≥ 80% line coverage on services.
Backend integration	Jest + Supertest + Testcontainers (MySQL)	Full HTTP round-trips against a real MySQL instance per test file. Covers auth, guards, transactional writes, file uploads.
Frontend unit	Vitest + React Testing Library	Pure components and hooks (form validation, editor state, blog filter).
End-to-end	Playwright	Four smoke flows: admin login + publish article; visitor browses home → service page → contact; visitor submits the contact form; admin reads and archives a message.
Accessibility	axe-core (in Playwright)	Automated WCAG checks on the 7 key public pages, blocking on PR.
Performance	Lighthouse CI	Runs against staging on each merge to develop; fails the build if Performance or Accessibility drops below 90 on the 3 main pages (home, telephony, contact).

Layer	Tool	What is tested
Manual exploratory	Test session checklist	Pre-release session on a real device for the rich-text editor and the responsive breakpoints (480/768/1024/1440 px).

CI / CD pipeline

GitHub Actions runs three workflows:

1. On every PR: install, lint, typecheck, unit + integration tests, build. Required to merge.
2. On merge to develop: deploy backend + both frontends to the staging environment, run Playwright + Lighthouse CI against staging.
3. On tag push (vX.Y.Z) from main: deploy backend + frontends to production, generate the client-facing release notes from the CHANGELOG.

Environments

Env	Backend host	Database	Frontend host
dev	localhost	Docker MySQL	Vite dev servers (5173 public, 5174 admin)
staging	VPS (1 instance) or Fly.io	Managed MySQL (small)	Same host, separate domains (staging.homni.fr, admin.staging.homni.fr)
prod	VPS or Fly.io (≥ 1 instance)	Managed MySQL with daily backups	homni.fr, admin.homni.fr — both behind Cloudflare

6.3 Definition of Done

A user story is Done when:

- Code is merged into develop through a reviewed PR.
- Unit and integration tests covering the new behaviour are present and green.
- If the story is user-visible, a Playwright E2E or a documented manual test has been executed against staging.
- API changes are reflected in openapi.yaml and the dashboard's generated client compiles.
- If the story touches the schema, a migration is included and the CI migrate / drop / migrate cycle passes.
- On user-facing pages, Lighthouse Accessibility ≥ 90 and axe-core has zero violations of severity ≥ serious.
- Sentry is consulted on staging: no new errors of severity ≥ error within 24h after deploy.

7. Technical Justifications

Every major technology choice is justified below against the functional requirements (FR), non-functional requirements (NFR) and team constraints (C) defined in Stages 1 and 2.

Choice	Driving need	Justification
React + Vite	C: mandated stack (Holberton). FR: two frontends, fast iteration.	Vite gives near-instant HMR and a fast production build, both important for a team of three working in parallel. File-based routing and an SPA fallback fit the dual-app structure perfectly.
Static export for the public site	NFR: SEO and Lighthouse Performance ≥ 90 (Stage 2 objective).	Pre-rendering each blog article to plain HTML eliminates first-paint JavaScript on the most-visited surface (the blog) — required to clear Lighthouse's TTI and LCP budgets on slow mobile networks.
TypeScript everywhere	NFR: maintainability across three developers and twelve weeks.	Catches a whole class of bugs at compile time and gives the dashboard autocomplete on the API DTOs once they are shared.
NestJS	C: mandated stack (Holberton). NFR: clean module boundaries.	Built-in DI, modules, guards, interceptors and pipes match the layered design used here. The OpenAPI module provides /docs for free.
MySQL 8	FR: strong relations between articles, categories, authors, media. NFR: a battle-tested engine with simple operations.	The domain is intrinsically relational and the volume is modest (a few hundred articles, a few hundred contact messages). MySQL is widely understood and trivially backed up — important for a project that will be handed off to the client.
TypeORM	NFR: schema versioned in code; type-safe queries.	Decorators-based entities pair naturally with NestJS modules. Migrations live in Git and travel with the code.
argon2id for passwords	NFR: industry-standard security.	Modern memory-hard hash, OWASP-recommended over bcrypt for new projects. Tunable cost lets us raise the bar if needed.
JWT access (15 min) + opaque refresh (30 d)	FR: admin stays signed in. NFR: token revocation possible.	Short-lived JWTs avoid hitting the DB on every request; refresh tokens are stored hashed so a DB leak does not yield usable tokens.
Local /uploads + daily rsync backup	FR: cover images persisted. NFR: minimal moving parts for a < 200 images / year volume.	An S3-compatible bucket would be overkill at this scale and adds a vendor relationship. The rsync-to-bucket strategy gives off-site backup without the live dependency.
Plausible (over Google Analytics)	NFR: GDPR compliance with minimal banner friction.	Plausible's aggregate-only analytics do not require explicit consent under CNIL guidance, so the cookie banner stays narrow. Google Analytics would force a more intrusive consent flow.
Sentry	NFR: visibility on real	Free tier sufficient for the MVP; SDKs for Nest,

Choice	Driving need	Justification
	user errors on day one.	React and Vite; release tagging integrates with our deployment script.
GitHub Actions	NFR: CI co-located with the code; no separate vendor.	Already where the source lives; generous free minutes on private repos; integrates with branch-protection rules.
Jest + Supertest + Testcontainers	NFR: tests that exercise real DB behaviour.	Avoids the false confidence of mocked repositories — many bugs (constraint violations, cascade behaviour, encoding) only surface against a real engine.
Playwright for E2E	NFR: cross-browser coverage on the few critical flows.	Runs in Chromium, Firefox and WebKit from a single test file. Built-in tracing makes failures debuggable. Open-source, no SaaS dependency.
Cloudflare in front	NFR: HTTPS, basic DDoS protection, caching of static assets.	Free tier provides what we need (CDN, automatic HTTPS, basic WAF) and reduces traffic to the origin — useful both for cost and for the blog-list cache strategy.

7.2 Alternatives considered and rejected

Alternative	Considered for	Why rejected
Next.js (App Router) for the public site	Frontend framework	Would give SSR / ISR out of the box, but it would mean two different frontend frameworks in the monorepo (Next for public, Vite for dashboard). A pre-rendered Vite app delivers the same result for a static, content-rich site without the framework split.
WordPress for the public site + blog	Whole project	Would be faster to set up but defeats the project's purpose: demonstrating full-stack engineering. It would also lock Homni into a PHP stack they cannot maintain in-house.
Strapi / Directus headless CMS	Admin dashboard	Cuts the admin work in half, but again leaves nothing to demonstrate. It also imposes a CMS data model on us instead of letting us shape it from the user stories.
PostgreSQL	Data store	Equally valid and arguably more powerful, but introduces an engine the team has less collective experience with. MySQL's ubiquity at smaller hosts simplifies the eventual hand-off to Homni's hosting provider.
MongoDB	Data store	The data is fully relational (articles ↔ categories ↔ authors ↔ media). A document store would force denormalisation just to reach the same integrity guarantees.

Alternative	Considered for	Why rejected
Trunk-based development	SCM	Better for teams with continuous deployment and feature flags; here we want explicit, tagged client deliverables every few weeks, which Git Flow's release branches model naturally.
GraphQL	API style	Overhead doesn't pay off for two first-party clients with predictable query shapes. REST keeps the docs simple and lets us use HTTP caching headers on the public blog endpoints.

7.3 Risks and mitigations

Risk	Likelihood / impact	Mitigation
Real Homni content (text, photos, testimonials) arrives late	High / Medium	Identified in Stage 1. We request all content in week 1; otherwise we use realistic placeholders validated by Homni and swap them in before the final demo.
Design disagreements between team and Homni	Medium / Medium	Two Figma directions presented early in the project, one validated before integration begins.
Team member unavailable (illness, exams)	Medium / Medium	Mandatory code review and continuous documentation so anyone can pick up another's work. Three full-stack profiles = versatility.
Spam flood on the contact form	Medium / Medium	IP rate limit + honeypot field + server-side validation. If insufficient, we add an invisible reCAPTCHA in a later release.
Schema migration breaks production	Low / High	Migrations reviewed in PR. Backward-compatible expansion-then-contraction pattern (add nullable column → backfill → enforce NOT NULL in a later release).
SEO impact of switching domains	Medium / Medium	301 redirects from the old URLs to the new ones, deployed at cutover. Sitemap.xml submitted to Search Console the same day.

7.4 Out-of-scope decisions deferred

- Multi-region deployment — homni.fr's audience is Occitanie; a single EU region is enough.
- Online quote generation — Homni's sales cycle is consultative; defer until the team and the client agree on a structured offer catalogue.

- Multilingual (English) version — Homni's market is local; revisit only if commercial demand emerges.
- Newsletter and marketing automation — adds an email-list dependency; out of scope for v1.

HOMNI

Refonte du site web

Étape 3 — Documentation Technique

Site vitrine B2B et dashboard de gestion de contenu

React + Vite · NestJS · MySQL 8 · TypeORM

Équipe : Leny BLEE · Ethane ZIMMERMANN · Tom MARCHAL

Holberton School · Projet de groupe tutoré · 12 semaines

Client : Homni — Montauban / Toulouse, Occitanie

Version du document : 1.0 · Mai 2026

— *Version française* —

0. Introduction

0.1 Objet du document

Ce document constitue le livrable de l'Étape 3 du projet de refonte du site web Homni — projet de groupe tutoré mené à Holberton School. Il traduit les objectifs définis en Étape 1 (constitution d'équipe et choix du MVP) et en Étape 2 (planification haut niveau, planning 12 semaines, objectifs SMART) en un plan technique concret : user stories, architecture, modèle de données, diagrammes de séquence, surface API et processus (gestion de code, qualité, déploiement) qui encadreront l'implémentation.

Il sert de source de vérité unique pour la phase de développement. Toute décision technique prise pendant le build doit soit être cohérente avec ce document, soit être consignée comme une déviation explicite via une pull request qui met à jour ce document.

0.2 Le client et le produit en un paragraphe

Homni est un prestataire B2B basé à Montauban et Toulouse qui propose quatre services complémentaires aux entreprises d'Occitanie : téléphonie IP, infogérance (réseau, helpdesk, cybersécurité), solutions d'impression et affichage dynamique. Le site actuel ne reflète pas le positionnement de l'entreprise et n'offre aucun parcours permettant aux prospects de se qualifier et de demander un rendez-vous. La refonte le remplace par un site vitrine moderne et responsive présentant clairement les quatre pôles d'expertise, un blog public pour le content marketing, et un parcours de contact structuré. Elle dote également Homni d'un dashboard d'administration permettant à l'équipe éditoriale de publier des articles de blog de manière autonome, sans dépendre d'un développeur.

0.3 Périmètre du MVP couvert

Ce document couvre le MVP convenu en Étape 1 : le site vitrine public (7 pages clés — accueil, quatre pages services, à propos, blog liste + détail, contact), le dashboard admin pour le CRUD des articles de blog uniquement (les autres fonctionnalités éditoriales sont hors périmètre de cette itération), l'authentification JWT pour les utilisateurs admin, le formulaire de contact avec consentement RGPD, et le pipeline de déploiement. Le modèle de données est volontairement étroit : 6 tables. La roadmap pourra découper le build en phases, mais la conception doit supporter tout le MVP dès le départ car le schéma et l'API sont difficiles à refactoriser une fois le site public en ligne et indexé.

1. User Stories et Maquettes

1.1 Personas

Camille — la dirigeante de PME (prospect). Dirige une entreprise de 35 personnes à Montauban et va perdre sa ligne téléphonique analogique. Ne connaît pas la différence entre un trunk SIP et un PBX hébergé. A besoin que le site explique l'offre en français clair, que Homni inspire confiance comme partenaire local, et de pouvoir prendre RDV en moins de deux minutes.

Marc — DSI d'une entreprise de 200 personnes (prospect averti). Compare 3 prestataires pour un renouvellement d'infogérance. Maîtrise déjà le vocabulaire technique. Veut des références concrètes, des articles de blog récents qui prouvent l'expertise de Homni, et des coordonnées claires pour un appel d'avant-vente technique.

Sophie — responsable marketing chez Homni (utilisatrice admin). Chargée de publier 2 à 3 articles par mois. N'est pas développeuse ; habituée aux éditeurs façon WordPress. A besoin de rédiger, sauvegarder, prévisualiser, attacher une image de couverture et publier sans solliciter l'agence.

Pierre — directeur commercial chez Homni (utilisateur admin léger). Se connecte une à deux fois par semaine pour lire les messages de contact reçus et répondre par email. A besoin que le dashboard soit évident et rapide depuis un navigateur de laptop.

1.2 User stories priorisées (MoSCoW)

Les stories portent un identifiant unique (US-xx) pour leur traçabilité avec les endpoints API, les diagrammes de séquence et les cas de test.

Must Have

ID	Persona	Story	Priorité
US-01	Camille / Marc	En tant que visiteur, je veux arriver sur une page d'accueil qui dit clairement ce que fait Homni et pour qui, afin de décider en 10 secondes si je reste.	Must
US-02	Camille / Marc	En tant que visiteur, je veux une page dédiée pour chacun des 4 services (téléphonie, infogérance, impression, affichage) afin d'approfondir celui qui me concerne.	Must
US-03	Camille	En tant que visiteur, je veux un formulaire de contact avec un sélecteur de sujet clair afin de demander des informations sans téléphoner.	Must
US-04	Camille	En tant que visiteur, je veux que le site charge et soit réactif sur mon poste de bureau et sur mon téléphone afin de ne pas rebondir.	Must
US-05	Camille / Marc	En tant que visiteur, je veux que le site soit entièrement responsive afin de pouvoir naviguer depuis n'importe quel appareil, y compris une tablette de 5 ans.	Must

ID	Persona	Story	Priorité
US-06	Marc	En tant que visiteur averti, je veux lire un blog avec des articles techniques catégorisés afin d'évaluer l'expertise de Homni.	Must
US-07	Marc	En tant que visiteur, je veux que chaque article de blog ait une mise en page propre et lisible avec image, titre, corps et date de publication.	Must
US-08	Sophie	En tant qu'éditrice, je veux me connecter à un dashboard privé avec mon email et mon mot de passe afin de gérer le contenu.	Must
US-09	Sophie	En tant qu'éditrice, je veux créer, éditer, sauvegarder en brouillon et publier des articles de blog afin de livrer du contenu en autonomie.	Must
US-10	Sophie	En tant qu'éditrice, je veux attacher une image de couverture à chaque article afin que la liste du blog ait l'air professionnelle.	Must
US-11	Sophie / Pierre	En tant qu'éditrice, je veux choisir une catégorie pour chaque article (téléphonie, infogérance, impression, affichage) afin que le blog public puisse être filtré.	Must
US-12	Tous visiteurs	En tant que visiteur, je veux une bannière de consentement RGPD et une page de confidentialité afin que mes données soient traitées légalement.	Must

Should Have

ID	Persona	Story	Priorité
US-13	Pierre	En tant que commercial, je veux voir les messages du formulaire de contact dans le dashboard afin de ne pas avoir à fouiller dans ma boîte mail.	Should
US-14	Sophie	En tant qu'éditrice, je veux prévisualiser mon article avant publication afin de vérifier le rendu.	Should
US-15	Marc	En tant que visiteur, je veux filtrer les articles de blog par catégorie afin de trouver rapidement ce qui me concerne.	Should
US-16	Pierre	En tant que commercial, je veux marquer un message de contact comme lu ou archivé afin que le dashboard reste rangé.	Should
US-17	Tous visiteurs	En tant que visiteur, je veux un sitemap.xml et des meta tags corrects afin que le site soit bien indexé par Google.	Should

Could Have

ID	Persona	Story	Priorité
US-18	Sophie	En tant qu'éditrice, je veux programmer un article pour publication future afin de préparer un calendrier éditorial.	Could
US-19	Pierre	En tant que commercial, je veux un graphique simple des messages mensuels afin de suivre le volume de leads.	Could
US-20	Tous visiteurs	En tant que visiteur, je veux m'abonner à une newsletter depuis le blog afin de recevoir les nouveaux articles.	Could
US-21	Sophie	En tant qu'éditrice, je veux une médiathèque pour réutiliser des images entre articles afin de ne pas re-uploader deux fois le même fichier.	Could

Won't Have (cette itération)

- US-22 — Génération de devis en ligne. Le cycle de vente est consultatif ; un devis exige un appel. Hors MVP.
- US-23 — Espace client (factures, tickets). Doublonnerait les outils internes existants de Homni ; hors périmètre.
- US-24 — Site multilingue (version anglaise). Le marché de Homni est l'Occitanie ; reporté.
- US-25 — Commentaires sur les articles. Coût de modération disproportionné pour le trafic attendu.

1.3 Maquettes — écrans principaux

Trois écrans concentrent l'essentiel de l'expérience : la page d'accueil publique (première impression), la liste de blog (surface de content marketing) et l'éditeur d'article admin (le poste de travail principal de l'éditrice). Les wireframes sont volontairement de basse fidélité afin de focaliser l'équipe sur la mise en page et la hiérarchie de l'information plutôt que sur le design visuel — le style final sera défini dans Figma au début de l'Étape 4.

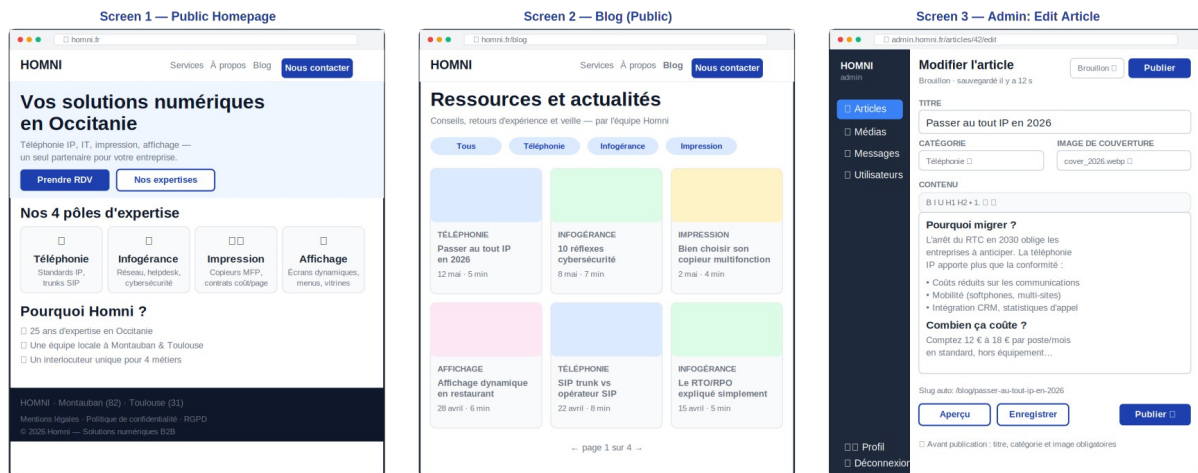


Figure 1 — Wireframes des trois écrans principaux : page d'accueil publique (US-01 à US-05), liste de blog avec filtres par catégorie (US-06, US-15), éditeur d'article admin (US-09, US-10, US-11, US-14).

Écran 1 — Page d'accueil publique

- Navigation haute : logo + 3 liens (Services, À propos, Blog) + un CTA contrasté "Nous contacter".
- Bloc hero énonçant la proposition de valeur en une phrase courte avec deux CTAs : "Prendre RDV" (primaire) et "Nos expertises" (secondaire).
- Ligne "4 pôles d'expertise" — une carte par service, chacune menant à sa page dédiée (US-02).
- Bloc de réassurance listant 3 à 4 faits rassurants (années d'expérience, présence locale, interlocuteur unique).
- Footer avec liens légaux (mentions légales, politique de confidentialité, RGPD) — obligatoire pour US-12.

Écran 2 — Liste de blog publique

- Titre de page et sous-titre d'une phrase posant les attentes ("Ressources et actualités par l'équipe Homni").
- Pills de filtre par catégorie (Tous, Téléphonie, Infogérance, Impression, Affichage) — US-15.
- Grille responsive de cartes d'articles (3 par ligne en desktop, 2 en tablette, 1 en mobile). Chaque carte affiche catégorie, titre, date et temps de lecture estimé.
- Pagination en bas (pas de scroll infini — meilleur pour le SEO et pour les navigateurs anciens).

Écran 3 — Éditeur d'article admin

- Sidebar gauche persistante : Articles, Médias, Messages, Utilisateurs.
- Header de l'éditeur : titre, statut de sauvegarde (auto-save toutes les 30 s), sélecteur de statut de publication, bouton primaire "Publier".
- Champs du formulaire : titre, catégorie, image de couverture (US-10), puis éditeur rich-text pour le corps. Le slug est généré depuis le titre et affiché en lecture seule.
- Barre d'actions basse : Aperçu (US-14), Enregistrer (en brouillon), Publier.
- Message de validation inline rappelant à l'éditrice que titre, catégorie et image de couverture sont obligatoires avant publication.

2. Architecture du Système

2.1 Diagramme global

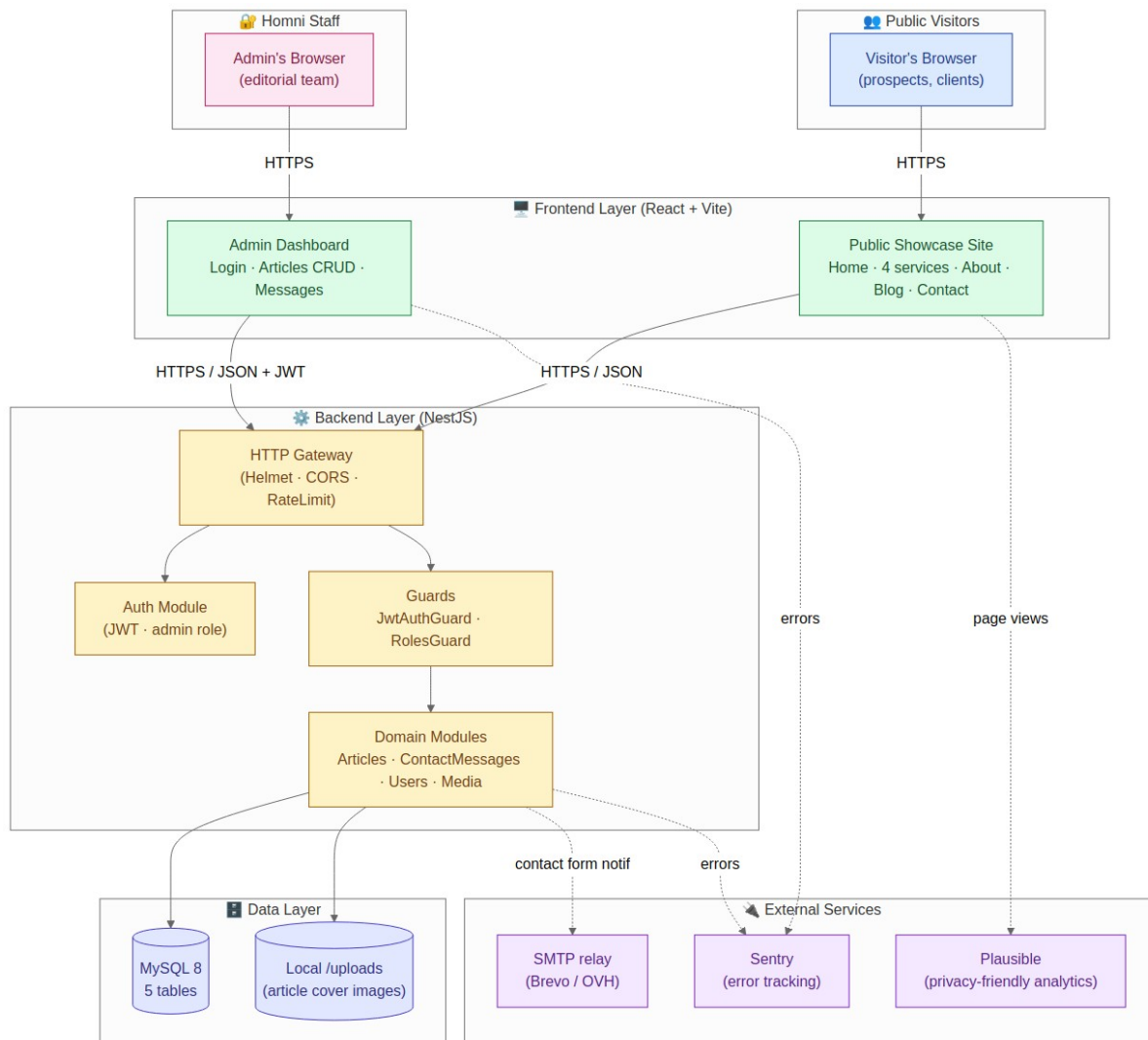


Figure 2 — Diagramme de composants. Flèches pleines = HTTP / JSON synchrone. Flèches pointillées = flux asynchrones ou différés (notifications de formulaire de contact, error tracking, analytics).

2.2 Responsabilités par couche

Couche frontend — React + Vite

Le projet est un monorepo TypeScript avec deux applications React qui partagent un package de composants UI. Le site vitrine public est rendu en statique au build (export statique via Vite + une petite étape de pré-rendu pour le blog) afin que chaque page soit servie comme du HTML pur — critique pour le SEO et les scores Lighthouse (US-04, US-17). Le dashboard admin est une SPA classique servie derrière authentification et n'est pas indexée.

Couche backend — NestJS

NestJS structure l'API en modules métier (Auth, Users, Articles, Categories, Media, ContactMessages) avec injection de dépendances explicite. Les préoccupations transversales vivent au niveau du gateway : Helmet pour les headers de sécurité HTTP, une politique CORS restreinte aux deux origines frontend connues, et un rate limiter (plus strict sur /contact-messages pour absorber le spam, plus lâche sur les endpoints de lecture). Deux guards globaux (JwtAuthGuard, RolesGuard) imposent l'authentification et l'autorisation par rôle avant qu'un controller ne s'exécute.

Couche données — MySQL 8 et filesystem local

MySQL 8 stocke les données métier durables. Le schéma (6 tables, voir section 3) est géré par des migrations TypeORM et n'est jamais modifié directement en production. Les images de couverture des articles sont stockées sur le filesystem local du serveur sous /uploads avec des noms basés sur UUID déterministes ; un cron de backup rsync ce dossier quotidiennement vers un bucket externe. Un object store managé (S3-compatible) est une décision reportée : il n'est pas justifié au volume d'articles attendu (< 200 images / an) et ajoute de la complexité opérationnelle.

Services externes

- Relais SMTP (Brevo ou OVH SMTP) — utilisé uniquement pour notifier l'équipe Homni des nouveaux messages de contact. Aucun email transactionnel n'est envoyé aux visiteurs dans le cadre du MVP.
- Sentry — error tracking côté serveur et côté client avec tagging par release. Le tier gratuit suffit au trafic attendu.
- Plausible Analytics — analytics privacy-friendly. Choisi à la place de Google Analytics pour éviter la complication de la bannière de consentement RGPD pour des analytics purement agrégées.

2.3 Parcours d'une requête

Une requête typique d'écriture — « publier un article » — suit un parcours net à travers les couches :

1. Le dashboard admin envoie POST /articles avec le JWT de l'admin dans le header Authorization.
2. Le gateway applique les headers Helmet et valide que l'origine de la requête est celle du dashboard admin.
3. JwtAuthGuard vérifie la signature du token et attache { userId, role } à la requête.
4. RolesGuard vérifie que le rôle est admin ou editor — requis pour toute écriture sur /articles.
5. ArticlesController valide le body (DTO class-validator) et délègue à ArticlesService.
6. Le service génère le slug, exécute l'INSERT dans une transaction, et retourne la ligne créée.
7. La réponse est sérialisée par un interceptor Nest qui retire les champs internes et renvoie 201 avec le nouvel article.

Une requête typique de lecture — « afficher la liste de blog » — est plus courte : le site public fait GET /articles?status=published&category=... sans authentification, le gateway applique un rate limit généreux, et la réponse est mise en cache 5 minutes via un header Cache-Control.

L'invalidation du cache est implicite : à la prochaine publication, le nouveau contenu apparaît au plus tard 5 minutes après — acceptable pour un site marketing.

3. Composants, Classes et Modèle de Données

3.1 Diagramme entité-relation

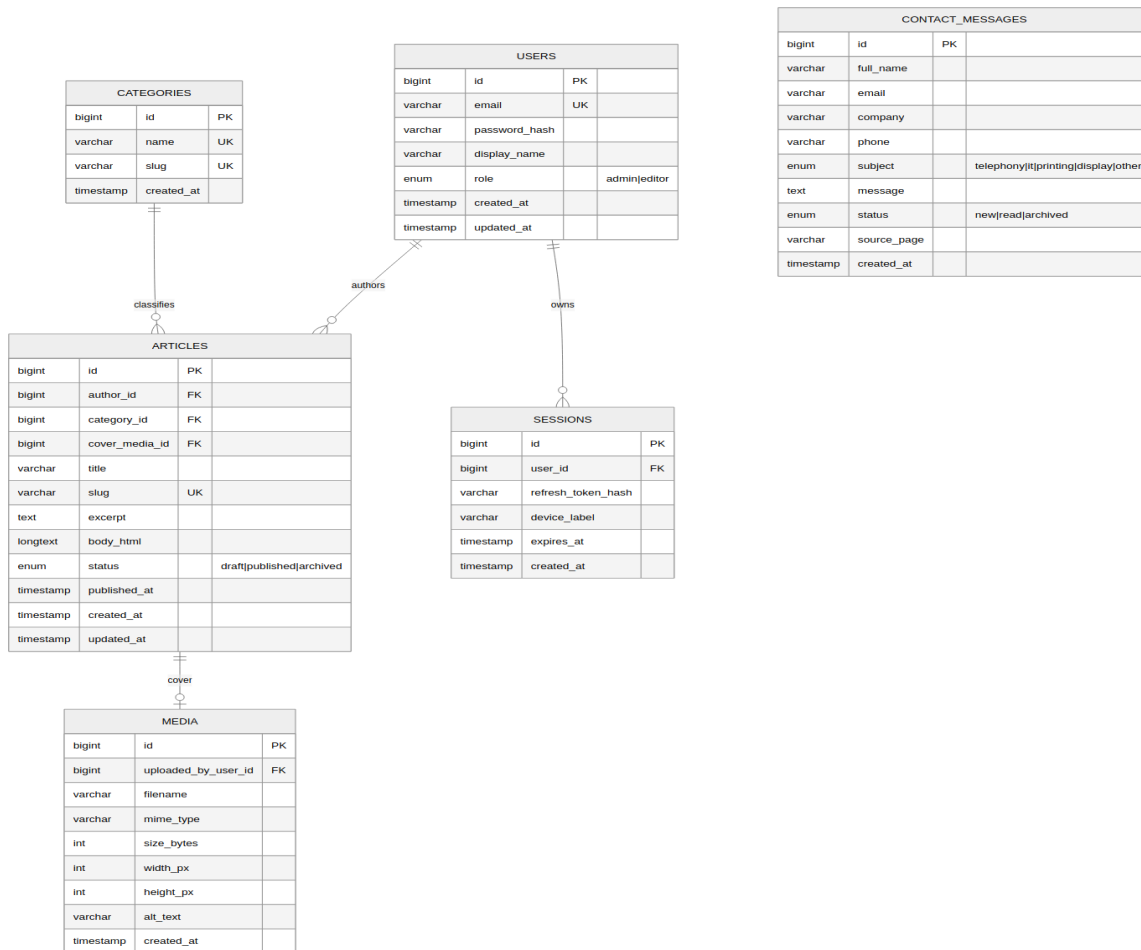


Figure 3 — Diagramme entité-relation. 6 tables, toutes en InnoDB avec clés primaires BIGINT surrogate. Uniques composites et index documentés en section 3.4.

3.2 Structure des classes backend

Chaque module métier suit la même forme — Module / Controller / Service / Repository / Entity / DTOs. Voici les classes clés avec leur responsabilité principale.

Classe	Responsabilité
UserService	Création de compte (admin uniquement), MAJ de profil, changement de mot de passe. Encapsule le hash (argon2id). Rôles : admin (accès complet) et editor (pas de gestion des utilisateurs).
AuthService	Émet les access tokens (15 min) et refresh tokens (30 jours, hashés en BDD). Vérifie les credentials. Déclenche les emails de reset (reporté en v1.1).
JwtAuthGuard	Guard global. Valide la signature et la fraîcheur du JWT, attache le contexte

Classe	Responsabilité
	utilisateur à la requête.
RolesGuard	Guard paramétrique. Pour les routes annotées @Roles('admin' 'editor'), vérifie le rôle de l'utilisateur.
ArticlesService	CRUD sur les articles. Génère le slug d'URL depuis le titre (slugify + check de collision). Impose les champs requis à la publication (titre, corps, catégorie, couverture).
CategoriesService	Catalogue en lecture seule exposant les quatre catégories métier fixes. Seedées dans une migration ; non éditables via le dashboard.
MediaService	Stocke les fichiers uploadés sous /uploads, valide MIME et dimensions, écrit une ligne media pointant vers le fichier. Les fichiers sont servis par Nest en assets statiques derrière une URL CDN-friendly.
ContactMessagesService	Persiste les soumissions du formulaire, fait la validation serveur et le check honeypot, déclenche une notification SMTP de manière asynchrone.
NotificationService	Wrapper léger autour de nodemailer utilisé par ContactMessagesService et (plus tard) les flux de reset de mot de passe. Le contenu des emails est templaté.

3.3 Schéma de base de données (MySQL 8)

Toutes les tables utilisent InnoDB avec utf8mb4 / utf8mb4_0900_ai_ci. Les clés primaires BIGINT surrogate sont auto-incrémentées. Les foreign keys cascaden en DELETE uniquement quand la relation est de type ownership (un article sans auteur existant n'a aucun sens et est bloqué par ON DELETE RESTRICT ; un article sans image de couverture conserve sa ligne, donc SET NULL est le bon comportement).

Table : users

Colonne	Type	Contraintes	Notes
id	BIGINT	PK, AUTO_INCREMENT	Clé surrogate.
email	VARCHAR(255)	UNIQUE, NOT NULL	Mis en minuscules à l'écriture.
password_hash	VARCHAR(255)	NOT NULL	argon2id.
display_name	VARCHAR(60)	NOT NULL	Affiché en signature des articles.
role	ENUM('admin','editor')	NOT NULL	admin = accès complet ; editor = articles + médias uniquement.
created_at	TIMESTAMP	DEFAULT	

Colonne	Type	Contraintes	Notes
		CURRENT_TIMESTAMP	
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	

Table : sessions

Colonne	Type	Contraintes	Notes
id	BIGINT	PK	
user_id	BIGINT	FK → users.id, ON DELETE CASCADE	
refresh_token_hash	VARCHAR(255)	NOT NULL	Hash argon2id ; indexé pour les lookups.
device_label	VARCHAR(120)	NULL	Résumé du user-agent.
expires_at	TIMESTAMP	NOT NULL	Fenêtre glissante 30 jours.
created_at	TIMESTAMP	DEFAULT NOW	

Table : categories

Colonne	Type	Contraintes	Notes
id	BIGINT	PK	
name	VARCHAR(60)	UNIQUE, NOT NULL	Téléphonie, Infogérance, Impression, Affichage.
slug	VARCHAR(60)	UNIQUE, NOT NULL	Fragment d'URL : telephonie, infogerance, impression, affichage.
created_at	TIMESTAMP	DEFAULT NOW	

Table : media

Colonne	Type	Contraintes	Notes
id	BIGINT	PK	
uploaded_by_user_id	BIGINT	FK → users.id, NULL	Null si l'uploader est supprimé ; l'image reste.
filename	VARCHAR(255)	UNIQUE, NOT NULL	Nom déterministe basé sur UUID.
mime_type	VARCHAR(80)	NOT NULL	Uniquement image/webp, image/jpeg, image/png.
size_bytes	INT	NOT NULL	Max 5 Mo imposé à l'upload.
width_px	INT	NOT NULL	Utilisé côté front pour éviter le layout shift.
height_px	INT	NOT NULL	
alt_text	VARCHAR(255)	NOT NULL	Obligatoire pour l'accessibilité (WCAG).
created_at	TIMESTAMP	DEFAULT NOW	

Table : articles

Colonne	Type	Contraintes	Notes
id	BIGINT	PK	
author_id	BIGINT	FK → users.id, ON DELETE RESTRICT	Un article survit aux uploaders, pas aux auteurs.
category_id	BIGINT	FK → categories.id, ON DELETE RESTRICT	
cover_media_id	BIGINT	FK → media.id, NULL ON DELETE SET NULL	Optionnel mais requis par la validation avant publication.
title	VARCHAR(180)	NOT NULL	
slug	VARCHAR(200)	UNIQUE, NOT NULL	Généré ; minuscules ; URL-safe.
excerpt	VARCHAR(280)	NOT NULL	Utilisé en liste de blog et en meta description.
body_html	LONGTEXT	NOT NULL	HTML sanitisé produit par l'éditeur rich-text.

Colonne	Type	Contraintes	Notes
status	ENUM('draft','published','archived')	DEFAULT 'draft'	Indexé pour les requêtes de liste filtrées.
published_at	TIMESTAMP	NULL	Renseigné quand le status passe à published.
created_at	TIMESTAMP	DEFAULT NOW	
updated_at	TIMESTAMP	ON UPDATE NOW	

Table : contact_messages

Colonne	Type	Contraintes	Notes
id	BIGINT	PK	
full_name	VARCHAR(120)	NOT NULL	
email	VARCHAR(255)	NOT NULL	Validé ; mis en minuscules.
company	VARCHAR(120)	NULL	
phone	VARCHAR(40)	NULL	
subject	ENUM('telephony','it','printing','display','other')	NOT NULL	Aligné sur le sélecteur du formulaire.
message	TEXT	NOT NULL	Max 2000 caractères côté formulaire.
status	ENUM('new','read','archived')	DEFAULT 'new'	Pilote la vue inbox du dashboard (US-13, US-16).
source_page	VARCHAR(120)	NULL	Page depuis laquelle le formulaire a été soumis, pour l'attribution.
created_at	TIMESTAMP	DEFAULT NOW	

3.4 Index clés (hors clés primaires)

- users.email — UNIQUE.
- articles.slug — UNIQUE.
- articles (status, published_at DESC) — pilote la requête de liste publique.

- articles (category_id, status, published_at DESC) — pilote les requêtes filtrées par catégorie.
- sessions.refresh_token_hash — index non unique pour lookup rapide.
- contact_messages (status, created_at DESC) — pilote l'inbox du dashboard.

3.5 Arbre de composants front-end

Deux applications React partagent un package de composants UI. Le routing du site public utilise des routes basées sur les fichiers (un composant par page) ; le dashboard admin utilise un router imbriqué avec un shell persistant.

Site public

Composant	Rôle
RootLayout	Header (logo + nav + CTA contact), outlet principal, footer, bannière de consentement RGPD.
HomePage	Hero, ligne des quatre services, bloc de réassurance, teaser des derniers articles, CTA final.
ServicePage	Une par service : hero, problématique, offre, mini-cas client, CTA. Quatre instances pilotées par un paramètre de route.
AboutPage	Histoire de l'entreprise, équipe, implantation géographique.
BlogListPage	Filtre par catégorie, grille paginée d'articles (US-06, US-15).
BlogArticlePage	Hero avec image de couverture, titre, signature, corps, boutons de partage (US-07).
ContactPage	Formulaire de contact (US-03) avec sélecteur de sujet, case de consentement RGPD, états succès / erreur.
LegalPages	Mentions légales, Politique de confidentialité, Cookies. Pages de contenu pur exigées par la loi.

Dashboard admin

Composant	Rôle
AdminShell	Sidebar + topbar + outlet. Impose une session authentifiée au niveau de la route.
LoginPage	Formulaire email + mot de passe (US-08). Redirige les utilisateurs authentifiés vers /articles.
ArticlesListPage	Tableau triable et filtrable de tous les articles (status, catégorie, auteur). Actions groupées.
ArticleEditorPage	Éditeur pour création + édition (US-09). Éditeur rich-text, aperçu du slug, picker d'image de couverture.

Composant	Rôle
MediaLibraryPage	Grille des médias uploadés avec possibilité d'upload, de renommage du alt text, de suppression des fichiers non utilisés.
ContactInboxPage	Inbox des messages du formulaire de contact (US-13). Marquer comme lu / archivé (US-16).
UsersPage	Admin uniquement. CRUD sur les utilisateurs (invitation par email, attribution du rôle).

4. Diagrammes de Séquence

Trois flux ont été sélectionnés comme critiques à valider. Ensemble, ils couvrent l'authentification, un chemin d'écriture authentifié avec upload de fichier, et un chemin d'écriture non authentifié ouvert au public — les trois catégories de risque de l'architecture.

4.1 Connexion admin (US-08)

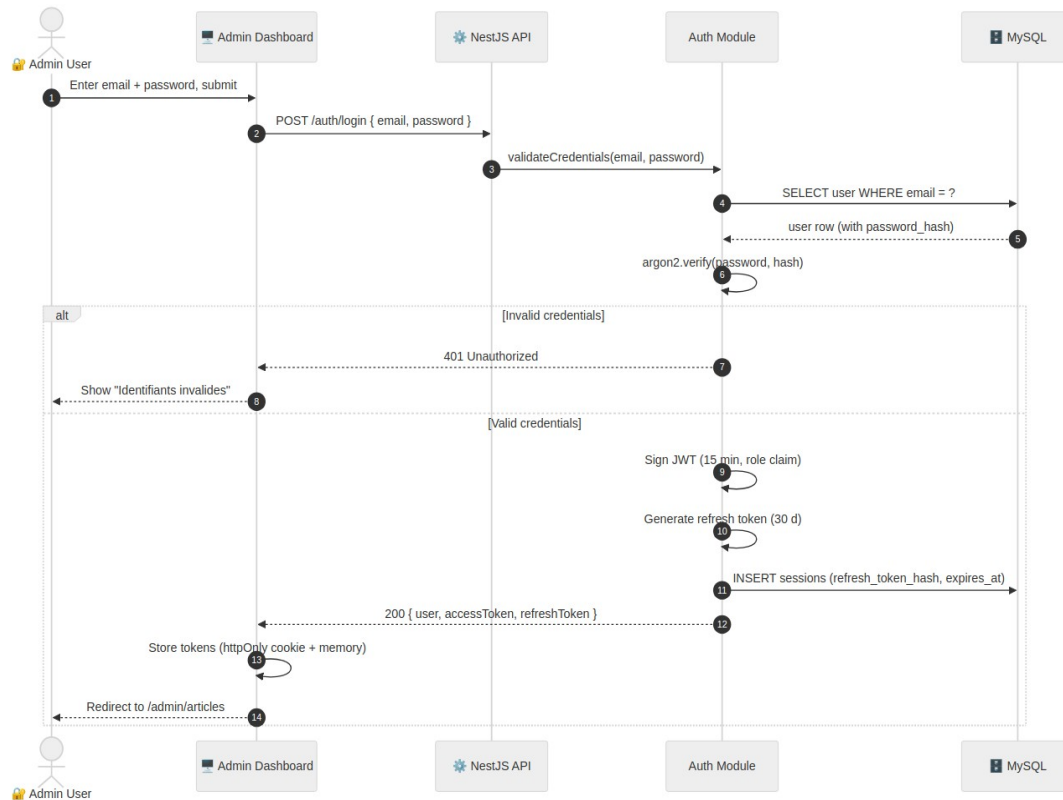


Figure 4 — Flux de connexion admin. Les deux branches (invalide + valide) sont documentées pour que le chemin d'échec soit testable.

Notes sur le flux :

- Les credentials sont vérifiés uniquement côté serveur. La comparaison de hash utilise argon2.verify, en temps constant et résistant aux attaques par timing.
- Le JWT porte l'id utilisateur et le rôle en claims, signé en HS256. Le dashboard utilise le claim de rôle pour afficher la bonne navigation, mais le backend ne lui fait jamais confiance sans re-vérifier via RolesGuard.
- Les refresh tokens sont stockés hashés (argon2id). Le token brut n'est envoyé qu'une fois dans la réponse de login et vit dans un cookie httpOnly côté dashboard.
- Une limite de tentatives (5 échecs par IP par 15 min) est appliquée au gateway pour mitiger les attaques brute-force.

4.2 Publier un article de blog (US-09, US-10, US-11)

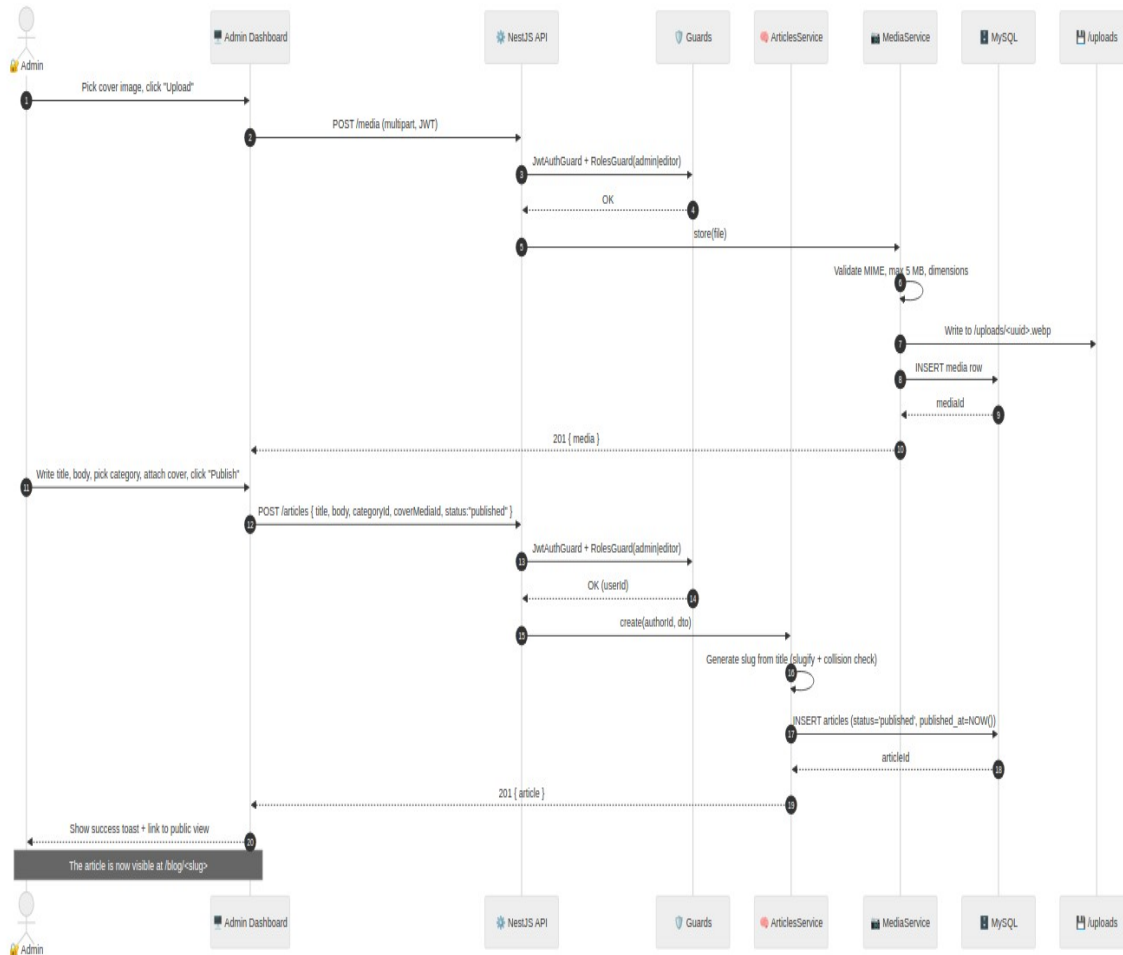


Figure 5 — Flux en deux étapes : upload du média d'abord, puis création de l'article. Les deux étapes sont découplées pour que la même image puisse être réutilisée sur plusieurs articles dans une future itération.

Notes sur le flux :

- Les uploads de fichiers sont validés pour MIME, taille et dimensions côté serveur, quelle que soit la validation côté client.
- Les fichiers uploadés sont renommés en UUID au stockage pour que le nom original n'apparaisse jamais dans l'URL publique.
- Les slugs sont générés par le service, pas par le client, pour garantir unicité et conformité URL.
- Une fois le status passé à published, l'article apparaît sur le site public au prochain rafraîchissement de cache (≤ 5 minutes).

4.3 Soumettre le formulaire de contact (US-03)

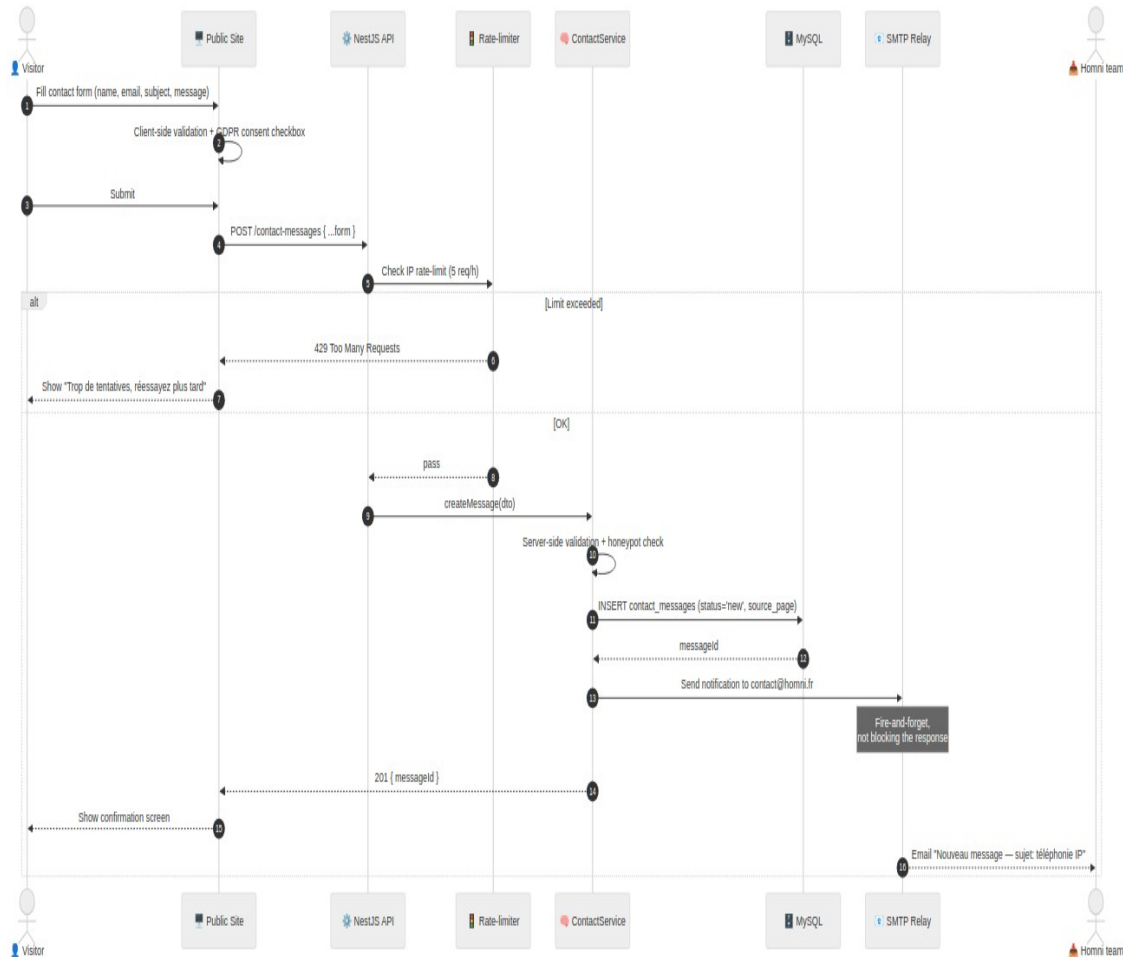


Figure 6 — Soumission du formulaire de contact. Le rate-limit et le honeypot absorbent ensemble la majorité du spam sans gêner les visiteurs légitimes.

- Le champ honeypot est un input caché que les humains ne remplissent jamais. Une valeur non vide déclenche un rejet silencieux (200 OK retourné pour ne pas signaler le piège aux bots).
- La notification SMTP est en fire-and-forget pour que la requête du visiteur revienne rapidement. Les échecs d'email sont loggés vers Sentry mais ne remontent pas à l'utilisateur.
- Le champ source_page est rempli par le client depuis le referrer pour que l'équipe commerciale puisse corréler les leads avec la page d'entrée.

5. Spécifications API

5.1 APIs externes consommées

Fournisseur	Usage	Justification
Relais SMTP (Brevo ou OVH)	Notifier Homni des nouveaux messages de contact	Les deux ont des tiers gratuits ou peu chers couvrant notre volume attendu (<200 messages/mois). SMTP laisse Homni maître de la boîte de destination sans nous enfermer dans une API vendor.
Sentry	Error tracking (backend + frontends)	Tier gratuit suffisant pour le MVP. Le tagging par release s'intègre à notre script de déploiement pour que chaque erreur référence le commit exact.
Plausible Analytics	Analytics privacy-friendly	Auto-hébergeable, pas de données personnelles, pas de bannière cookie requise pour l'analytics — simplifie la conformité RGPD. Choisi à la place de Google Analytics pour cette raison.

5.2 API REST interne — conventions

- Base URL : `https://api.homni.fr/v1`
- Toutes les requêtes et réponses utilisent `application/json; charset=utf-8` (sauf upload de média, en `multipart/form-data`).
- Les routes authentifiées exigent `Authorization: Bearer <accessToken>`. Routes publiques : `/articles` (lecture), `/categories`, `/contact-messages` (POST), `/auth/login`, `/auth/refresh`.
- Les erreurs suivent une forme inspirée de RFC 7807 : `{ statusCode, error, message, code }`.
- La pagination est offset-based sur le dashboard (`page + perPage`) car la donnée est petite ; la liste publique utilise aussi l'offset pour des URLs SEO-friendly (`?page=2`).
- Les dates sont en ISO-8601 avec timezone (le serveur est en UTC). Le client convertit pour l'affichage.
- Le versioning est dans le path (`/v1/...`). Un changement breaking incrémente vers `/v2` et tourne en parallèle pendant au moins un cycle de release.

5.3 Catalogue d'endpoints

La surface est volontairement étroite — environ 30 endpoints sur 6 modules — car le périmètre MVP est borné au blog et au contact. Les contrats input/output complets sont maintenus dans `openapi.yaml` et rendus via Swagger UI à `/docs` dans les environnements non-production.

Auth

Méthode	Path	Usage / réponse
POST	<code>/auth/login</code>	Connexion email + mot de passe. → 200 { user,

Méthode	Path	Usage / réponse
		accessToken, refreshToken }.
POST	/auth/refresh	Échanger un refresh token contre une nouvelle paire. → 200 { accessToken, refreshToken }.
POST	/auth/logout	Révoquer le refresh token courant. → 204.
GET	/auth/me	Retourner l'utilisateur authentifié. → 200 { user }.

Users (admin uniquement)

Méthode	Path	Usage / réponse
GET	/users	Lister tous les utilisateurs admin. → 200 [{ id, email, displayName, role }].
POST	/users	Inviter un nouvel utilisateur (envoie un email de reset). → 201 { user }.
PATCH	/users/:id	MAJ display name ou rôle. → 200 { user }.
DELETE	/users/:id	Soft-delete (impossible de supprimer le dernier admin). → 204.
PATCH	/users/me/password	Changer son propre mot de passe (exige l'actuel). → 204.

Categories (catalogue en lecture seule)

Méthode	Path	Usage / réponse
GET	/categories	Public. Retourne les 4 catégories fixes. → 200 [{ id, name, slug }].

Articles

Méthode	Path	Usage / réponse
GET	/articles?status=&categorySlug=&page=&perPage=	Public quand status=published ; sinon admin. → 200 { items, total, page }.
GET	/articles/:slug	Lookup public par slug pour la page article. → 200 { article }.

Méthode	Path	Usage / réponse
GET	/articles/id/:id	Lookup admin par id (utilisé par l'éditeur). → 200 { article }.
POST	/articles	Créer un article (brouillon par défaut). → 201 { article }.
PATCH	/articles/:id	MAJ des champs et/ou transition de status. → 200 { article }.
POST	/articles/:id/publish	Endpoint de confort : status='published', published_at=NOW. → 200 { article }.
POST	/articles/:id/archive	Archiver un article — caché de la liste publique. → 200 { article }.
DELETE	/articles/:id	Hard-delete (admin uniquement ; editor ne peut qu'archiver). → 204.

Media

Méthode	Path	Usage / réponse
GET	/media?page=&perPage=	Lister les médias uploadés (admin). → 200 { items, total, page }.
POST	/media	Uploader un nouveau fichier (multipart). → 201 { media }.
PATCH	/media/:id	MAJ du alt text. → 200 { media }.
DELETE	/media/:id	Supprimer (refusé si référencé par un article). → 204 ou 409.

Contact messages

Méthode	Path	Usage / réponse
POST	/contact-messages	Soumission publique depuis le formulaire. → 201 { messageId }.
GET	/contact-messages?status=	Vue inbox admin (US-13). → 200 { items }.
PATCH	/contact-messages/:id	MAJ du status à read ou archived (US-16). → 200 { message }.
DELETE	/contact-messages/:id	Hard-delete d'un message (admin uniquement). → 204.

5.4 Exemple requête / réponse

POST /contact-messages — un visiteur soumet le formulaire de contact :

```
POST /v1/contact-messages
Content-Type: application/json
```

```
{
  "fullName": "Camille Durand",
  "email": "c.durand@exemple.fr",
  "company": "Atelier Durand SAS",
  "phone": "+33 5 63 00 00 00",
  "subject": "telephony",
  "message": "Bonjour, nous souhaitons migrer notre standard en IP avant la fin de l'année.",
  "sourcePage": "/services/telephonie",
  "website": ""
}
```

Réponse (le champ website est le honeypot ; une valeur vide est normale) :

```
HTTP/1.1 201 Created
Content-Type: application/json
```

```
{
  "messageId": 318,
  "createdAt": "2026-05-25T14:09:51.027Z"
}
```

6. Plans SCM et QA

6.1 Stratégie de gestion de code

Le projet utilise un Git Flow allégé avec deux branches longues et des branches feature, release et hotfix éphémères. Cette stratégie a été préférée au trunk-based car (a) l'équipe est composée d'étudiants aux emplois du temps qui se chevauchent — une branche main stable protège le livrable pendant que le travail continue en parallèle sur develop — et (b) le client souhaite des versions discrètes et taguées (v1.0.0, v1.1.0...) liées à un périmètre identifiable, ce que les branches release modélisent naturellement.

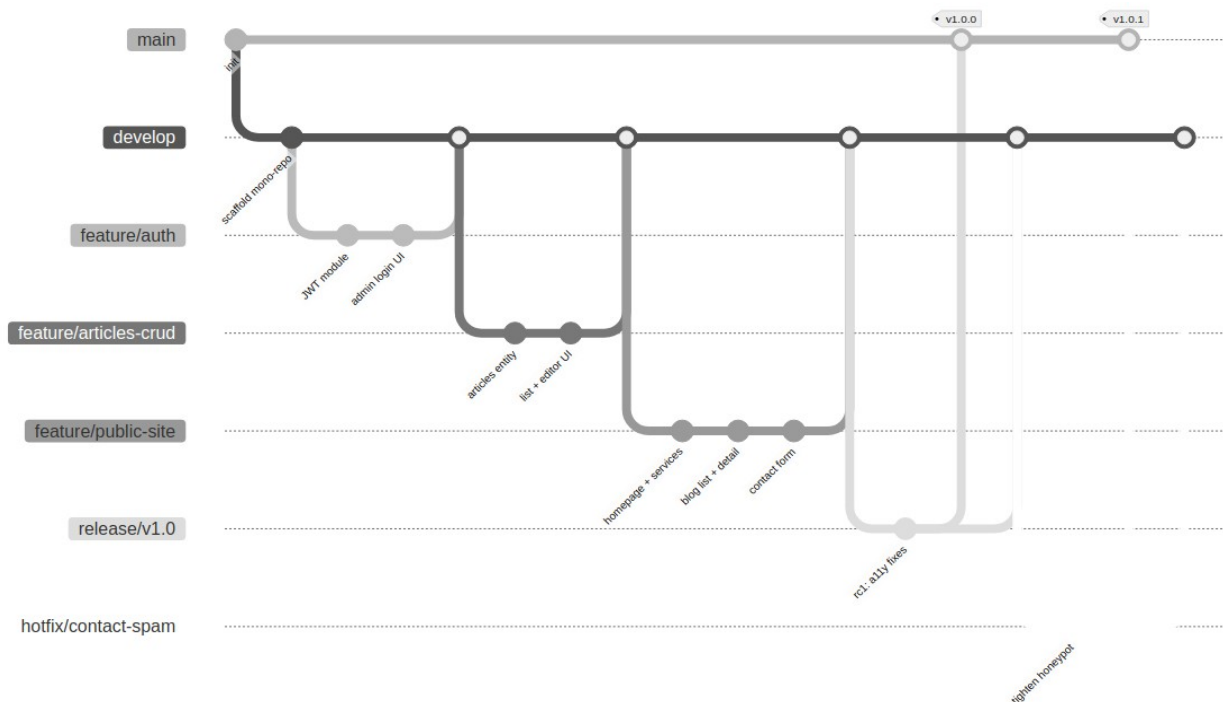


Figure 7 — Modèle de branches. main est toujours égal à la dernière release taguée ; develop est la branche d'intégration ; les branches feature sont courtes (≤ 1 semaine) ; les hotfix partent de main et sont mergés dans main et develop.

Branches

- main — code production-ready. Chaque merge est tagué (vX.Y.Z) et déploie en production.
- develop — branche d'intégration. Toutes les branches feature y sont mergées. L'environnement de staging auto-déploie depuis develop.
- feature/<short-slug> — une branche par user story ou tâche technique. Partie de develop, mergée via PR reviewée.
- release/vX.Y.Z — ouverte quelques jours avant une démo client pour le polish de dernière minute. Mergée dans main et develop.
- hotfix/<slug> — fixes d'urgence partis de main, mergés dans main et develop.

Conventions de commit et de PR

- Format Conventional Commits : type(scope): subject — types : feat, fix, chore, docs, refactor, test, ci, perf.
- Les PR exigent au moins un build CI vert, une suite de tests verte, une approbation de reviewer, et une checklist self-review (pas de code commenté, pas de console.log, migrations incluses si pertinent).
- Les titres de PR référencent l'ID de user story — ex. "feat(articles): publish flow (US-09)".
- Le squash-merge est le défaut pour garder un historique develop linéaire ; les merges release / hotfix vers main utilisent des merge commits pour préserver les tags.
- Branch protection sur main : pas de push direct, signed commits imposés.

6.2 Stratégie d'assurance qualité

La qualité est imposée à trois niveaux — statique, automatisé, manuel — chacun choisi pour attraper une classe de bug différente au stade le moins coûteux possible.

Couche statique (sans exécuter de code)

- TypeScript en mode strict sur le backend et le frontend.
- ESLint + Prettier avec une config partagée dans tout le monorepo. La CI échoue sur les erreurs de lint, pas seulement sur les warnings.
- class-validator sur chaque DTO. Les bodies invalides sont rejetés au gateway avec un 400 avant d'atteindre un service.
- Migrations BDD versionnées dans Git ; la CI vérifie que le schéma local drop + re-migrate correspond au dump de develop.
- axe-core exécuté dans la suite de tests du site public pour faire échouer les PR qui introduisent des violations WCAG sur les 7 pages clés (soutient l'objectif d'accessibilité de l'Étape 2).

Tests automatisés

Couche	Outil	Ce qui est testé
Unitaire backend	Jest	Méthodes de service (génération de slug, contrôle de rôle, transitions de status). Repository mocké. Cible : $\geq 80\%$ de couverture de lignes sur les services.
Intégration backend	Jest + Supertest + Testcontainers (MySQL)	Round-trips HTTP complets contre une vraie instance MySQL par fichier de test. Couvre auth, guards, écritures transactionnelles, uploads de fichiers.
Unitaire frontend	Vitest + React Testing Library	Composants purs et hooks (validation de formulaire, state de l'éditeur, filtre de blog).
End-to-end	Playwright	Quatre flows fumée : login admin + publier un article ; visiteur parcourt accueil → page service → contact ; visiteur soumet le formulaire ; admin lit et archive un message.
Accessibilité	axe-core (dans Playwright)	Checks WCAG automatisés sur les 7 pages publiques clés, bloquants en PR.

Couche	Outil	Ce qui est testé
Performance	Lighthouse CI	Exécuté contre staging à chaque merge vers develop ; fait échouer le build si Performance ou Accessibilité passe sous 90 sur les 3 pages principales (accueil, téléphonie, contact).
Exploratoire manuel	Checklist de test session	Session pré-release sur device réel pour l'éditeur rich-text et les breakpoints responsive (480/768/1024/1440 px).

Pipeline CI / CD

GitHub Actions exécute trois workflows :

1. Sur chaque PR : install, lint, typecheck, tests unit + intégration, build. Requis pour merger.
2. Sur merge vers develop : déploiement backend + deux frontends en staging, exécution de Playwright + Lighthouse CI contre staging.
3. Sur push d'un tag (vX.Y.Z) depuis main : déploiement backend + frontends en production, génération des release notes client depuis le CHANGELOG.

Environnements

Env	Hôte backend	Base de données	Hôte frontend
dev	localhost	MySQL Docker	Serveurs dev Vite (5173 public, 5174 admin)
staging	VPS (1 instance) ou Fly.io	MySQL managé (small)	Même hôte, domaines séparés (staging.homni.fr, admin.staging.homni.fr)
prod	VPS ou Fly.io (≥ 1 instance)	MySQL managé avec backups quotidiens	homni.fr, admin.homni.fr — tous deux derrière Cloudflare

6.3 Definition of Done

Une user story est Done lorsque :

- Le code est mergé dans develop via une PR reviewée.
- Les tests unit et intégration couvrant le nouveau comportement sont présents et verts.
- Si la story est visible par l'utilisateur, un E2E Playwright ou un test manuel documenté a été exécuté contre staging.
- Les changements d'API sont reflétés dans openapi.yaml et le client généré du dashboard compile.
- Si la story touche au schéma, une migration est incluse et le cycle CI migrate / drop / migrate passe.
- Sur les pages user-facing, Lighthouse Accessibilité ≥ 90 et axe-core a zéro violation de sévérité ≥ serious.
- Sentry est consulté en staging : pas de nouvelles erreurs de sévérité ≥ error dans les 24h suivant le déploiement.

7. Justifications Techniques

Chaque choix technologique majeur est justifié ci-dessous contre les exigences fonctionnelles (FR), non fonctionnelles (NFR) et les contraintes d'équipe (C) définies en Étapes 1 et 2.

Choix	Besoin moteur	Justification
React + Vite	C : stack imposée (Holberton). FR : deux frontends, itération rapide.	Vite offre un HMR quasi instantané et un build de production rapide, tous deux importants pour une équipe de trois qui travaille en parallèle. Le routing fichier et le fallback SPA collent à la structure dual-app.
Export statique pour le site public	NFR : SEO et Lighthouse Performance ≥ 90 (objectif Étape 2).	Pré-rendre chaque article de blog en HTML pur élimine le JavaScript de premier paint sur la surface la plus visitée — requis pour tenir les budgets TTI et LCP de Lighthouse sur réseau mobile lent.
TypeScript partout	NFR : maintenabilité à trois développeurs sur douze semaines.	Attrape une classe entière de bugs à la compilation et donne au dashboard de l'autocomplete sur les DTO de l'API une fois ceux-ci partagés.
NestJS	C : stack imposée (Holberton). NFR : frontières de modules nettes.	DI, modules, guards, interceptors et pipes built-in collent à la conception en couches utilisée ici. Le module OpenAPI fournit /docs gratuitement.
MySQL 8	FR : relations fortes entre articles, catégories, auteurs, médias. NFR : moteur éprouvé et opérations simples.	Le domaine est intrinsèquement relationnel et le volume est modeste (quelques centaines d'articles, quelques centaines de messages de contact). MySQL est largement connu et trivialement sauvegardable — important pour un projet qui sera transmis au client.
TypeORM	NFR : schéma versionné dans le code ; requêtes type-safe.	Les entités à base de décorateurs se marient naturellement aux modules NestJS. Les migrations vivent dans Git et voyagent avec le code.
argon2id pour les mots de passe	NFR : sécurité standard industrie.	Hash memory-hard moderne, recommandé OWASP face à bcrypt pour les nouveaux projets. Coût ajustable pour relever la barre si besoin.
JWT access (15 min) + refresh opaque (30 j)	FR : l'admin reste connecté. NFR : révocation possible.	Les JWT courts évitent un hit BDD à chaque requête ; les refresh tokens stockés hashés signifient qu'une fuite BDD ne donne pas de tokens exploitables.
/uploads local + backup rsync quotidien	FR : images de couverture persistées. NFR : peu de pièces mobiles pour un volume < 200 images / an.	Un bucket S3-compatible serait overkill à cette échelle et ajoute une relation vendor. La stratégie rsync-to-bucket fournit du backup off-site sans la dépendance live.
Plausible (plutôt	NFR : conformité RGPD	Les analytics agrégées de Plausible n'exigent pas

Choix	Besoin moteur	Justification
que Google Analytics)	avec friction de bannière minimale.	de consentement explicite selon les recommandations de la CNIL, ce qui garde la bannière étroite. Google Analytics imposerait un flux de consentement plus intrusif.
Sentry	NFR : visibilité sur les erreurs utilisateur réelles dès le jour 1.	Tier gratuit suffisant pour le MVP ; SDKs pour Nest, React et Vite ; tagging par release intégré à notre script de déploiement.
GitHub Actions	NFR : CI co-localisée avec le code ; pas de vendor séparé.	Déjà là où vit le code ; minutes gratuites généreuses sur repo privé ; s'intègre aux règles de branch protection.
Jest + Supertest + Testcontainers	NFR : des tests qui exercent le vrai comportement BDD.	Évite la fausse confiance des repositories mockés — beaucoup de bugs (violations de contraintes, cascades, encodage) n'apparaissent que face à un vrai moteur.
Playwright pour l'E2E	NFR : couverture cross-browser sur les quelques flows critiques.	Tourne en Chromium, Firefox et WebKit depuis un seul fichier de test. Le tracing intégré rend les échecs debuggables. Open-source, pas de dépendance SaaS.
Cloudflare en frontal	NFR : HTTPS, protection DDoS basique, cache des assets statiques.	Le tier gratuit couvre ce qu'il nous faut (CDN, HTTPS automatique, WAF basique) et réduit le trafic vers l'origine — utile pour le coût et pour la stratégie de cache de la liste de blog.

7.2 Alternatives envisagées et rejetées

Alternative	Considérée pour	Pourquoi rejetée
Next.js (App Router) pour le site public	Framework frontend	Donnerait SSR / ISR out of the box, mais imposerait deux frameworks frontend différents dans le monorepo (Next pour public, Vite pour dashboard). Une app Vite pré-rendue donne le même résultat pour un site statique riche en contenu sans le split de framework.
WordPress pour le site public + blog	Tout le projet	Plus rapide à mettre en place mais défait l'objectif du projet : démontrer l'ingénierie full-stack. Verrouillerait aussi Homni dans une stack PHP qu'ils ne peuvent pas maintenir en interne.
CMS headless (Strapi / Directus)	Dashboard admin	Coupe le travail admin en deux, mais là encore ne laisse rien à démontrer. Impose aussi un modèle de données CMS au lieu de nous laisser le façonner depuis les user stories.
PostgreSQL	Stockage	Tout aussi valable et sans doute plus puissant, mais introduit un moteur dont l'équipe a moins d'expérience collective. L'ubiquité de MySQL

Alternative	Considérée pour	Pourquoi rejetée
		chez les petits hébergeurs simplifie la transmission éventuelle à l'hébergement de Homni.
MongoDB	Stockage	La donnée est totalement relationnelle (articles ↔ catégories ↔ auteurs ↔ médias). Un document store imposerait de la dénormalisation pour atteindre les mêmes garanties d'intégrité.
Trunk-based development	SCM	Mieux pour les équipes en CD avec feature flags ; ici nous voulons des livrables client explicites et tagués toutes les quelques semaines, ce que les branches release de Git Flow modélisent naturellement.
GraphQL	Style d'API	Le surcoût ne paie pas pour deux clients first-party aux formes de requête prévisibles. REST garde la doc simple et permet l'usage des headers de cache HTTP sur les endpoints publics du blog.

7.3 Risques et mitigations

Risque	Probabilité / impact	Mitigation
Le contenu réel Homni (textes, photos, témoignages) arrive en retard	Élevée / Moyen	Identifié en Étape 1. Tout le contenu est demandé en semaine 1 ; sinon nous utilisons des placeholders réalistes validés par Homni et les remplaçons avant la démo finale.
Désaccord design entre l'équipe et Homni	Moyenne / Moyen	Deux directions Figma présentées tôt dans le projet, l'une validée avant que l'intégration ne commence.
Indisponibilité d'un membre (maladie, examens)	Moyenne / Moyen	Code review obligatoire et documentation continue : chacun peut reprendre le travail d'un autre. Trois profils full-stack = polyvalence.
Vague de spam sur le formulaire de contact	Moyenne / Moyen	Rate limit par IP + champ honeypot + validation côté serveur. Si insuffisant, ajout d'un reCAPTCHA invisible dans une release ultérieure.
Migration de schéma casse la prod	Faible / Élevé	Migrations reviewées en PR. Pattern backward-compatible expand-then-contract (ajout colonne nullable → backfill → enforce NOT NULL dans une release ultérieure).
Impact SEO du changement	Moyenne / Moyen	Redirections 301 des anciennes

Risque	Probabilité / impact	Mitigation
de domaine		URLs vers les nouvelles, déployées au cutover. sitemap.xml soumis à la Search Console le même jour.

7.4 Décisions hors périmètre reportées

- Déploiement multi-région — l'audience de homni.fr est en Occitanie ; une seule région EU suffit.
- Génération de devis en ligne — le cycle de vente de Homni est consultatif ; à reporter jusqu'à ce que l'équipe et le client s'accordent sur un catalogue d'offres structuré.
- Version multilingue (anglais) — le marché de Homni est local ; à revoir uniquement si une demande commerciale émerge.
- Newsletter et marketing automation — ajoute une dépendance à une liste d'emails ; hors périmètre v1.